Bounded Rescheduling for Flexible Job Shops via Spatiotemporal Reservation Closure

QI YU, Southeast University, PR China

WEI LI*, Southeast University, PR China

In flexible job shop scheduling that captures processing order with an operation-precedence graph (operation order within each job and processing order on each machine) and records the intervals in which each vehicle occupies each corridor in a time-window reservation table, the standard way to delineate the rescheduling scope after a disturbance is still affected-operation rescheduling (AOR): it first identifies the operations that can no longer be executed as planned because of the disturbance, and then adds their successors along the operation-precedence graph. Disturbances such as corridor blockage or corridor slowdown act only on corridors; machines and operations remain available, so AOR finds no affected operation, returns an empty rescheduling scope, and concludes that no rescheduling is needed. In fact, the planned corridor occupancies that overlap the disturbed interval are already invalid and the original schedule is infeasible. Moreover, since a corridor admits only one vehicle at a time, a vehicle planned to traverse it later must wait for the preceding vehicle to leave (hereafter, a yielding relation); once the preceding vehicle is delayed, the following one is delayed in turn, and the delay passes from vehicle to vehicle along yielding relations. The problem is not that the disturbance goes undetected (the blockage itself is known), but that the operation-precedence graph on which AOR delineates the scope contains only operations and their precedence relations; it has neither corridor occupancies nor the yielding relations among them, and therefore cannot express this kind of invalidation or its propagation.

We instead delineate the rescheduling scope at the level of corridor occupancies and propose the spatiotemporal reservation closure (STRC); the set of occupancies it outputs is called the reservation impact set. It is constructed as follows: starting from the planned corridor occupancies whose occupancy intervals overlap the disturbed interval, it follows four kinds of dependency relations among occupancies—yielding on the same corridor, the next occupancy of the same vehicle, the occupancy of the next operation of the same job, and the occupancy of the next operation on the same machine—and keeps adding successor occupancies until no new occupancy can be added (i.e., it takes the transitive closure). In the original plan, each corridor occupancy is determined by three items: the corridor traversed (i.e., the path), the occupancy interval, and the carrying vehicle. Occupancies inside the impact set may have their path and interval replanned (and, under disturbances such as vehicle failure, their vehicle replaced); occupancies outside it keep all three items of the original plan. Once the four kinds of dependency relations are fixed, whether an occupancy belongs to the reservation impact set depends only on whether it is reachable from the seeds along these relations; the result is unique and does not depend on which rerouting algorithm is used afterwards. Bounded repair on the impact set has two levels: it first keeps the original vehicle assignment and replans conflict-free paths only for the occupancies inside the set; if the schedule is still infeasible, it successively adds the subsequent occupancies of the same job, the subsequent occupancies of the same vehicle, and finally all occupancies unfinished at the decision time to the rewritable scope, retrying at each step.

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Experiments use 10 random seeds on each of 5 instances (the random seed determines disturbance injection and the search process), giving 50 instance–seed pairs. Under corridor blockage, the rescheduling scope delineated by AOR on the operation-precedence graph is empty on all 50 pairs, although the original schedule is in fact infeasible on every one of them. With the rerouting procedure unchanged and only the scope object switched from the operation-precedence graph to the reservation impact set, the number of pairs on which repair yields a feasible schedule rises from 0/50 to 50/50. On all 50 pairs the reservation impact set satisfies two properties: first, it is closed, i.e., no dependency edge leads from an occupancy inside the set to one outside it; second, after repair, the corridor, interval, and vehicle of every occupancy outside the set coincide with the original plan. If, with everything else unchanged, the rewritable set is replaced by "all occupancies unfinished at the decision time", runtime and makespan remain almost unchanged, but the fraction of rewritten occupancies rises from 0.540 to 0.604: among the 50 pairs, scoping by the reservation impact set rewrites fewer, equally many, or more occupancies on 31/18/1 pairs, respectively (two-sided Wilcoxon signed-rank test, $p = 8.8 \times 10^{-7}$). We also compare with warm-started global rescheduling, which takes the original schedule as the initial solution, re-searches machine assignment and processing order within a given time limit, and likewise does not rewrite finished occupancies. Bounded repair is two to three orders of magnitude faster, but its makespan $C_{\max}$ (the completion time of all jobs) is never better than that of global rescheduling under any tested time limit (0.2–2 s); since global rescheduling already wins at the shortest time limit and its $C_{\max}$ can only decrease as the limit is relaxed, bounded repair cannot overtake it under longer limits either.

Hence, the restoration of feasibility comes from replacing the scope object: with the rerouting procedure unchanged, switching the scope object from the operation-precedence graph to the reservation impact set restores feasibility. The reduction in rewrites comes from the reservation impact set being smaller than "all unfinished occupancies"; in our measurements, however, the former already covers about 92% of the latter on average, excluding less than one tenth of the occupancies, so the reduction is limited, and on instances where the impact set approaches all unfinished occupancies the two nearly coincide and the reduction disappears. Rerouting inside the set, keeping the original plan outside it, and enlarging the rewritable scope after a failed repair all follow existing local rescheduling practice; the contribution of this paper lies in re-determining which kind of object the rescheduling scope should be delineated on, namely replacing the set of operations with the reservation impact set formed by corridor occupancies.

CCS Concepts: • **Theory of computation** → **Scheduling algorithms**; • **Applied computing** → *Industry and manufacturing*; • **Computing methodologies** → *Planning and scheduling*.

Additional Key Words and Phrases: flexible job shop scheduling, bounded rescheduling, conflict-free AGV routing, spatiotemporal reservation table, spatiotemporal reservation closure, corridor blockage

ACM Reference Format:
Qi Yu and Wei Li. 2026. Bounded Rescheduling for Flexible Job Shops via Spatiotemporal Reservation Closure. 1, 1 (November 2026), 43 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

1.1 Background and motivation

Flexible job shop scheduling with transportation usually takes the transport time of a job between two machines as a constant indexed by location pairs [13, 16, 25], implicitly assuming that vehicles never compete for corridors. This assumption breaks down when a finite fleet shares a sparse corridor network [5], so existing work records the intervals in which each vehicle occupies each corridor in a time-window reservation table before execution and decides machine assignment jointly with conflict-free paths [7, 9, 20, 23]. A schedule then contains, besides machine assignment and processing order, a time-window reservation table recording which vehicle occupies which corridor during which interval. A disturbance may make operations unexecutable as planned, e.g., a machine failure or a vehicle failure; or it may invalidate only some corridor occupancies in the reservation table without changing any operation, e.g., a corridor being blocked or slowed down for some period.

These works answer how to build a conflict-free schedule before execution, when information is complete. Once the shop has executed up to time t_{now} and a disturbance occurs, three questions arise: which corridor occupancies the disturbance invalidates; which occupancies must be rewritten so that the schedule again satisfies corridor exclusivity and the closure constraints under the disturbance and can thus continue to be executed; and whether such an executable schedule can be obtained within milliseconds of computation time without restarting a population-based search.

1.2 The gap the operation-precedence graph cannot capture

In dynamic scheduling of flexible job shops with transportation, the rescheduling scope is usually delineated on the operation-precedence graph. Specifically, the operations that can no longer be executed as planned because of an equipment failure serve as seeds; the operations that follow the seeds within the same job, and the operations processed after the seed operations on the same machine, are added to the scope. Alternatively, the expansion may proceed from the seeds along the graph for at most θ steps, where θ is a manually set upper bound on the expansion depth that keeps the rescheduling scope near the seeds and prevents it from spreading along dependency chains over the whole schedule. Only the operations in this scope are then rescheduled, and all other operations keep the original plan. In the existing literature, dynamic scheduling studies with AGV transport and with machine failures or rush orders as disturbances delineate the scope in this way. Their recovery scopes are, respectively, "the operations not yet processed after the failure" [26] and "the operations and delivery tasks classified by disturbance impact into retained, continued, and reconstructed" [22] (the latter studies a hybrid flow shop with lot streaming whose disturbances include vehicle failures, but its model has no reservation table). This practice follows the idea of affected-operation rescheduling (AOR) [18]. In what follows, a baseline that delineates the scope on the operation-precedence graph represents this practice (Section 2.1).

This way of delineating the scope is often reasonable when the disturbance itself is an operation event, e.g., a machine failure that prevents several operations from being executed under the original assignment. When the schedule records corridor occupancies with time windows, however, there is another class of disturbances: those that make a corridor impassable for some period without disabling any machine or operation. A typical example is corridor blockage, in which a corridor is closed to traffic during $[t_a, t_b)$. A blockage modifies neither machine assignment nor processing order, and machines and operations remain available, so the seed set of AOR—the operations that can no longer be executed as planned because of the disturbance—is empty, and expansion along the operation-precedence graph has nowhere to start; the delineated scope is empty, and the method concludes that no rescheduling is needed. Yet the planned corridor occupancies that overlap the blockage interval conflict with the closure, so the original schedule is infeasible. Moreover, a corridor is an exclusive resource that admits only one vehicle at a time, so a vehicle planned to traverse the same corridor later must wait for the preceding vehicle to leave; we call this precedence a yielding relation. When the blocked vehicle is delayed by waiting or detouring, the vehicle planned to traverse the corridor after it is delayed in turn, which then delays its own follower, so the invalidation propagates vehicle by vehicle across the reservation table along yielding relations.

"Cannot capture" in the title of this subsection means the following: the blockage itself is known, so the problem is not detection; rather, the operation-precedence graph on which AOR delineates the scope has operations as vertices and job and machine precedences as edges, and by definition contains neither corridor occupancies nor their yielding relations, so it can express neither the occupancy invalidation nor its propagation between vehicles. If transport times are still taken from a constant matrix indexed by location pairs, corridors are not contested resources and no reservation

table exists, so this kind of invalidation cannot even be expressed in the model; the gap therefore arises only in settings that record corridor occupancies with time windows.

Accordingly, we distinguish two classes of disturbances.

- **Class A** (acting on the operation-precedence graph): robotic-arm failures, significant deviations in processing time, rush-order insertions, and *vehicle failures*. These disturbances prevent some operations from being executed as planned, so the seed operation set used to delineate the rescheduling scope is nonempty; scoping on the operation-precedence graph and scoping on the reservation impact set both yield nonempty scopes, so this class is not the focus of our tests. The reason for placing vehicle failures in Class A is given in Section 3.3: although a vehicle failure changes no operation assignment, it makes the operations carried by that vehicle unexecutable, so the mapping from a vehicle to the operations it carries still yields a nonempty seed operation set on the operation-precedence graph.
- **Class B** (acting only on the reservation table): corridor blockage, corridor slowdown, temporary revocation of local right of way, and the like. These disturbances disable no machine or operation, so at the operation level no operation appears affected and the seed operation set on the operation-precedence graph is empty; yet the set of corridor occupancies that overlap the disturbed interval is nonempty, and so is the reservation impact set obtained by taking them as seeds.

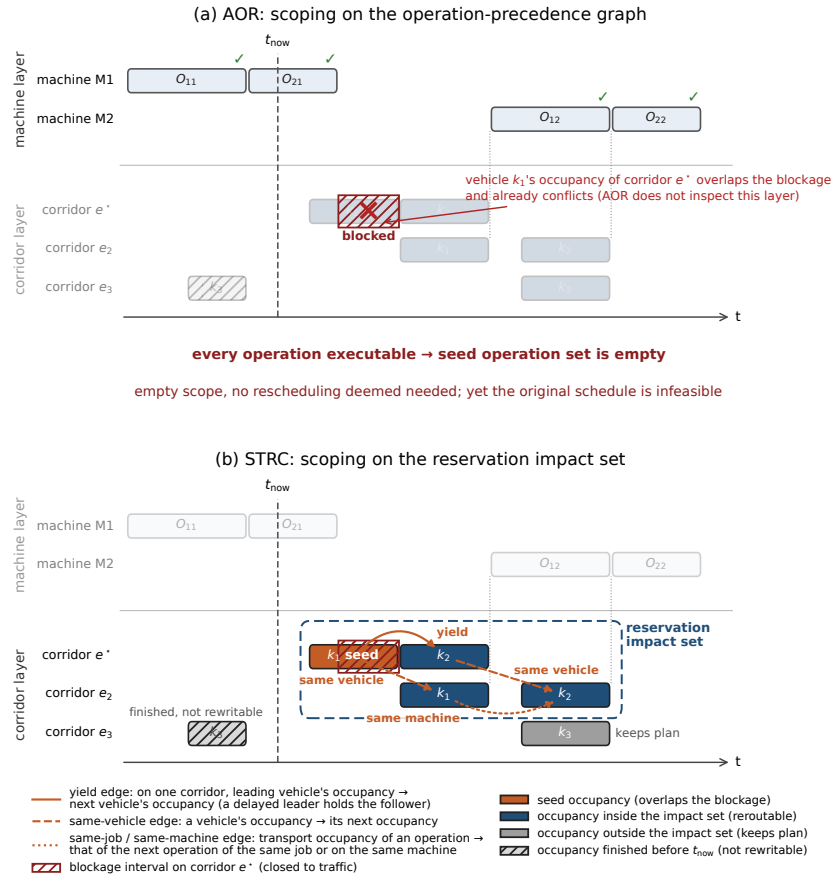
Hence, when a model captures processing order with the operation-precedence graph and records corridor occupancies with time windows, a rescheduling scope delineated only on the operation-precedence graph is empty under a Class B disturbance. The existing dynamic scheduling studies with AGVs cited above do not consider corridor contention among vehicles (Section 2.4), so the operation-precedence graph is the only scope object available to them; the problem arises only when a model contains both the operation-precedence graph and a corridor reservation table (Section 2.5). Figure 1 illustrates the gap. In panel (a), the scope is delineated on the operation-precedence graph, which contains no seed operation, so no rescheduling is deemed necessary; in panel (b), the scope is delineated on the reservation table by the spatiotemporal reservation closure proposed in this paper (Section 4): the occupancies that overlap the blockage interval become seeds, and subsequent occupancies are added step by step along yielding relations and succession relations (the next occupancy of the same vehicle, of the same job, and on the same machine), forming the reservation impact set.

1.3 Claims

This paper makes three claims.

First, when a flexible job shop guarantees conflict-free vehicle movement through time-window reservations, the rescheduling scope after a disturbance should be defined on the reservation dependency graph formed by corridor occupancies and their yielding relations, not only on the operation-precedence graph.

Second, the scope is delineated by the spatiotemporal reservation closure (STRC), whose result is called the reservation impact set. Starting from the corridor occupancies that overlap the disturbed interval and are therefore invalid, it follows yielding, the next occupancy of the same vehicle, the occupancy of the next operation of the same job, and the occupancy of the next operation on the same machine, and includes in the rescheduling scope every corridor occupancy that is thereby invalidated, directly or indirectly, and must be given a new path or time window. Once these dependencies are fixed, no occupancy in the set has a dependency edge pointing to an occupancy outside it, so "which occupancies to rewrite" corresponds to a verifiable set rather than a manually chosen search radius.



Notation: O_{ji} is operation i of job j; M1, M2 are machines; k_1, k_2, k_3 are vehicles (k_1 carries job 1, k_2 carries job 2, k_3 carries another job); $e^\star$, e_2, e_3 are corridors; each bar in the corridor layer is one occupancy of a corridor by a vehicle, and a thin dotted line links it to the operation it delivers to.

Fig. 1. Motivating example: the same schedule under the same corridor blockage, inspected at different layers by the two scoping methods. In each panel the upper layer is the machine layer (operation Gantt chart) and the lower layer is the corridor layer (reservation table); the hatched box is the blockage interval on corridor $e^\star$, and the dashed vertical line is the decision time t_{now}. (a) AOR inspects only the machine layer: every operation is executable, the seed operation set is empty, and no rescheduling is deemed needed; yet vehicle k_1's occupancy on the corridor layer already conflicts with the blockage, so the original schedule is infeasible. (b) The proposed method inspects the corridor layer: vehicle k_1's occupancy overlapping the blockage becomes the seed, and subsequent occupancies are added along the yield edge ($k_1 \rightarrow k_2$, same corridor), same-vehicle edges, and the same-machine edge, forming the reservation impact set (dashed box); occupancies outside it keep the original plan, and occupancies finished before t_{now} are not rewritable.

Third, once the impact set is obtained, the original vehicles are rerouted inside the set while occupancies outside it are kept; if no feasible schedule results, the rewritable scope is enlarged in the order of the same job, the same vehicle, and finally all unfinished occupancies. For Class A failures, reassigning vehicles and machines is additionally allowed, but the processing order is still kept. These steps follow existing practice and serve to act on the impact set and restore a feasible schedule.

The three claims share one stance: **the reservation impact set answers only "which occupancies are already invalid", not "how the schedule should be changed".** We use the dependencies among occupancies only to decide which occupancies are already invalid: an occupancy reachable from the seeds along the dependencies belongs to the impact set, and one that is not reachable does not. These relations are not used to guide the search or to change assignments and paths; that is a decision problem of schedule improvement and lies outside the scope of this paper.

Which dependency edges to include is a modeling choice; once the edge set is chosen, the closure of the impact set under outgoing edges (no occupancy in the set has a dependency edge leading outside it) follows from the definition (Proposition 2). The impact set is not necessarily the minimal set of occupancies needed to restore feasibility, because the minimal set depends on the permitted repair actions, whereas the definition of the impact set is not tied to any particular repair method (Remark 2).

Compared with global rescheduling that starts from the original schedule and searches again within a time limit, the reservation impact set addresses a different question: when an executable plan must be produced quickly, can feasibility be restored while confining rewrites to the impact set? As long as the repair first writes the occupancies outside the impact set back into the new reservation table unchanged and the final schedule passes the conflict check, the corridor, interval, and vehicle of every corridor occupancy outside the impact set coincide with the original plan (Proposition 3). When the scope is delineated on the operation-precedence graph, a corridor blockage triggers no local rescheduling because no affected operation is found, and the original schedule remains infeasible; triggering global rescheduling instead restores feasibility, but requires a search time of seconds: on instances with 8 jobs and 4 machines, the search time limits we test are 0.2–2 s, and the smallest limit allows only about one generation to be evaluated; the time of a single decoding grows with instance size, and at 20 jobs, 10 machines, and 10 vehicles evaluating a single generation of the population already takes about 1.4 s (Section 6.11). Compared with global rescheduling (which likewise does not rewrite occupancies already finished at the decision time), bounded repair has a rewrite fraction of the same order and a much shorter runtime (milliseconds versus seconds), but on all 12 instance–seed pairs measured, global rescheduling attains a smaller $C_{\max}$ under every search time limit. Short runtime and fewer rewrites must be tested separately (Section 6.9): the short runtime comes from planning one path for each transport leg in the impact set without restarting a population-based search, not from the impact set itself; whether the impact set reduces rewrites cannot be inferred from the comparison with global rescheduling (the two rewrite fractions are already close), and requires a separate baseline that uses the same rerouting procedure and differs only in whether the rewritable set is the impact set or all unfinished occupancies. This paper therefore focuses on identifying the invalidated corridor occupancies, and on the corresponding local recovery, when feasibility must be restored immediately.

1.4 Contributions

The contributions of this paper are as follows; they are referred to as C1, C2, and C3.

C1. Identifying a class of invalidation that the operation-precedence graph cannot capture. When a schedule captures processing order with operation precedence and records corridor occupancies with time windows, the

standard rescheduling scope is still delineated by the affected operations. Disturbances such as corridor blockage and corridor slowdown change no operation assignment, so the scope delineated by this method is empty, and the method concludes that no rescheduling is needed; yet the corridor occupancies that overlap the disturbed interval are already invalid, the original schedule is infeasible, and the resulting waits propagate to downstream vehicles through yielding relations. This holds in all 50 corridor-blockage pairs (50/50, Section 6.2). The root cause is that the operation-precedence graph contains no yielding relations among occupancies.

C2. Defining the rescheduling scope as the reservation impact set. Starting from the corridor occupancies that overlap the disturbed interval, the scope follows yielding, the next occupancy of the same vehicle, and the occupancies of the next operation of the same job and on the same machine, and includes every occupancy that is thereby invalidated, directly or indirectly, and must be given a new path or time window; this delineation method is called the spatiotemporal reservation closure (STRC). Once the dependencies are fixed, membership of an occupancy in the impact set is uniquely determined, and no occupancy in the set has a dependency edge leading outside it, so "what to rewrite" corresponds to a verifiable set rather than a manually chosen search radius and does not depend on how the rerouting algorithm is implemented. Experimentally, across four disturbance types and two scoping methods, the scope delineated on the operation-precedence graph is empty only under corridor blockage and corridor slowdown; the reservation impact set delineated by our method is nonempty under all four types and, under robotic-arm and vehicle failures, contains the former, so switching to the reservation impact set never misses an occupancy already included by the existing method (Section 6.8). The benefit of the delineation itself is fewer rewrites: with the same rerouting procedure, compared with not delineating and rewriting all unfinished occupancies, scoping by the impact set rewrites fewer occupancies (fewer, equally many, and more on 31/18/1 of the 50 pairs, respectively; Section 6.9).

C3. Providing a local recovery matched to this delineation and separating the sources of each result. The original vehicles are rerouted inside the impact set while occupancies outside it are kept; if the schedule remains infeasible, the rewritable scope is enlarged. The repair steps follow existing practice; this contribution lies in using controlled comparisons to separate the sources of each result. Under the same corridor network, time-window routing, and conflict-free validation, in the 50 corridor-blockage pairs, scoping on the operation-precedence graph yields an executable schedule in none of them (0/50), whereas scoping on the reservation impact set yields one in all of them (50/50). Hence whether an executable schedule is obtained depends on the scope object, not on the quality of the rerouting procedure. Bounded repair takes milliseconds of computation, but its makespan $C_{\max}$ is no better than that of global rescheduling under every computation time limit tested (0.2–2 s), and relaxing the limit does not reverse this order (Section 6.7). Compared with the non-delineating approach above, scoping on the impact set has almost identical runtime and $C_{\max}$, showing that the millisecond runtime comes from single-pass rerouting without restarting a population-based search, not from the impact set (Section 6.9).

In the same shop and on the same reservation table, integrated scheduling before execution uses the reservation table to build a conflict-free plan [7, 20, 23]; this paper uses the reservation table at execution time to locate the invalidated scope. The two act at different stages.

1.5 Organization

Section 2 reviews related work in four strands: how job shops delineate the rescheduling scope by operations or on the time axis, how railway scheduling represents the order of occupancies on exclusive resources, how conflict-free

multi-vehicle routing performs local replanning, and where flexible job shops with transportation draw the boundary; it then explains how this paper differs from existing work. Section 3 presents the corridor-reservation setting and the execution-time recovery problem: the shop, the corridor reservation table and the operation-precedence graph, the two classes of disturbances, and the conditions that recovery must satisfy. Section 4 defines the spatiotemporal reservation closure and its result, the reservation impact set, and establishes its dependency edges and properties such as closure under outgoing edges. Section 5 describes how the original vehicles are rerouted inside the impact set while occupancies outside it are kept, how the rewritable scope is enlarged when the schedule remains infeasible, and how this compares with global rescheduling. Section 6 reports feasibility, runtime, the rewrite fraction of occupancies, and makespan, and uses a case Gantt chart to show, on the operation time axis, the pattern of changes behind the statistical results. Section 7 concludes and discusses limitations and future work.

2 Related work

A basic question in dynamic scheduling is how to delineate the rescheduling scope: after a disturbance, which operations, corridor occupancies, or vehicles must have their original arrangements replanned, while all arrangements outside the scope keep the original plan. On which kind of object the rescheduling scope should be delineated, the literature takes different approaches: operations or the time axis, the occupancy order on exclusive resources, the affected vehicles, and the set of operations in the flexible job shop with transportation. We review, for each of these four classes, what has been done and what remains uncovered, and in Table 1 compare them item by item with this paper in terms of problem setting and rescheduling scope.

2.1 Local rescheduling in job shops

For operation-level events such as machine breakdowns and rush-order insertions, delineating the rescheduling scope by the affected operations is still common in dynamic scheduling with AGVs. Zhang et al. [26] handle machine breakdowns and rush orders, taking as the recovery scope the operations not yet processed after the breakdown. Tang et al. [22] classify operations and delivery tasks, according to the impact of the disturbance, as retained, continued, or reconstructed. They compare full rescheduling, partial rescheduling, and right shift side by side, noting that right shift preserves stability at the cost of optimization room. This practice is represented by affected-operation rescheduling (AOR) and its modification MAOR [18], which reschedule only the operations directly or indirectly affected by the disturbance. Delineating the scope along the time axis and enlarging it after failure have also been studied. Match-up methods search forward from the disturbance for a time point at which the original schedule can be resumed, and reschedule only the operations before it: Qiao et al. [15] apply this to a semiconductor manufacturing system and compare it with right-shift rescheduling; Salido et al. [17] use match-up to determine the rescheduling zone in a job shop with variable machine speeds and then reduce energy consumption within that zone. Local Rescheduling [6] enlarges the time window incrementally after a failed repair. Right shift, in contrast, does not determine which operations are affected; it postpones the entire unfinished part. Rolling-horizon methods split a long-horizon problem into overlapping shorter-horizon subproblems solved in sequence [8], and decomposition methods use the precedence structure to split the job set into blocks that are sequenced one after another [11]; see [14, 24] for surveys.

Delineating a rescheduling scope and then rescheduling locally within it is thus a fairly general practice in dynamic scheduling; all of the works above, however, start the delineation from disturbed operations. Once a disturbance only makes a corridor impassable without changing any assignment, there is no disturbed operation to start from, so the scope delineated by this approach is necessarily empty, even though the original schedule is already infeasible.

2.2 Occupancy order on exclusive resources

Modeling "who occupies the same exclusive resource first" as a graph has been studied systematically in railway scheduling. The alternative graph of Mascis and Pacciarelli [12] generalizes the disjunctive graph: vertices are occupations of block sections by operations, fixed arcs encode route precedence, and paired alternative arcs encode which train passes the section first. D'Ariano et al. [1] use this model for real-time train rescheduling, recomputing a conflict-free timetable after a disturbance while staying as close as possible to the original plan. An alternative arc captures the order in which two trains pass the same block section; the yielding relation between two vehicles passing the same corridor in this paper is the same kind of occupancy order on an exclusive resource.

In the alternative graph, each conflict corresponds to a pair of mutually exclusive arcs (A passes first, or B passes first), and scheduling selects one arc from each pair so that the graph contains no positive-weight cycle, the timetable is feasible, and the objective is optimized as far as possible; occupancy order is therefore a decision variable to be optimized. The railway model treats block sections as machines, so train runs and section occupancies live on the same graph, and an unavailable section shows up directly as an event on that graph; hence the problem of "a scope delineated on the operation-precedence graph missing a corridor blockage" does not arise there. This paper, by contrast, uses such relations after the plan has been scheduled and the order fixed: it does not choose the order again, but only follows the established yielding and succession relations to determine which corridor occupancies must be replanned after the disturbance.

2.3 Local replanning in conflict-free multi-vehicle routing

In multi-agent path finding (MAPF), replanning only the affected agents while treating the others as dynamic obstacles is standard practice. For online MAPF, Švancara et al. [21] propose Replan All, Replan Single, and Replan Affected, and determine the affected set by online independence detection. Malka et al. [10] likewise replan only the vehicles affected by observed changes when the map is inaccurate. Conflict-free path planning treats spatiotemporal occupancy as a first-class constraint, but start and goal locations are usually given externally [27].

These works have only vehicle start and goal locations and no job–operation–machine assignment, so the situation in which "the rescheduling scope is delineated on the operation-precedence graph and a corridor blockage is missed for lack of an affected operation" cannot arise. As for how the affected set is obtained, MAPF finds the affected vehicles by independence detection; the result is the output of a detection algorithm and varies with its implementation and stopping condition. The reservation impact set proposed here differs: once the dependency edges are given, whether an occupancy belongs to the set depends only on whether it is reachable from the seeds along these edges, independently of the rerouting algorithm used afterwards (Section 4.2).

2.4 Flexible job shop scheduling with transportation

Flexible job shop scheduling with transport vehicles has been systematically surveyed [25]. Recent combinatorial optimization work still commonly takes travel times as a constant matrix indexed by location pairs, and builds constraint programming and knowledge-guided search on top of it [13, 16]; other work lets transport times depend on load and approximates interaction between vehicles by an aggregate congestion-delay term [4]. Neither convention records which road segment a vehicle occupies and when, so a corridor blockage has no counterpart in the model and cannot be tested.

Table 1. Comparison of related work and this paper in problem setting and rescheduling scope. ✓: present; ✗: absent; —: not applicable (no local rescheduling scope is delineated). "Occupancy time windows" means that the model explicitly records the intervals during which each vehicle (train) occupies a segment (block section, spatiotemporal cell). "Scope on" is the object that may be rewritten inside the scope; "Propagation channels" are the channels tracked during delineation, named by the shared resource through which invalidation passes; "No misses" means that, under a disturbance acting only on corridors (e.g., a blockage or slowdown), all invalidated occupancies fall inside the scope; "Repair-independent" means that the scope is uniquely determined by a given rule and does not vary with the implementation or stopping condition of the repair or detection algorithm.

	Problem setting			**Rescheduling scope**			
Representative work	**Operation & machine assignment**	**Occupancy time windows**	**Execution-time disturbances**	**Scope on**	**Propagation channels**	**No misses**	**Repair-independent**
AOR [18]	✓	✗	Operation events	Affected operations	Job, machine	✗	✓
Match-up [15, 17], LRS [6]	✓	✗	Operation events	Time window	Window enlarged stepwise	✗	✗
Dynamic scheduling with AGVs [22, 26]	✓	✗	Equipment failures, rush orders	Operations and transport tasks	By disturbance category	✗	✓
Real-time railway rescheduling [1, 12]	✗	✓	Train delays	All trains	— (global rescheduling)	✓	—
Online MAPF [10, 21]	✗	✓	New tasks, map changes	Affected vehicles	Inter-vehicle path conflicts	✓	✗
Constant or aggregate transport time [4, 13, 16]	✓	✗	None (before execution)	—	—	—	—
Integrated conflict-free scheduling [7, 20, 23]	✓	✓	None (before execution)	—	—	—	—
This paper	✓	✓	**Operation events and corridor disturbances**	**Corridor occupancies (assignment and sequence unchanged)**	**Corridor, vehicle, job, machine**	✓	✓

Once corridor occupancies are recorded as time windows, both processing order and corridor occupancy enter the model. Congestion-aware dispatching can select vehicles using proxies such as local density, without changing machine assignments [5]. Work that jointly models flexible assignment and conflict-free transportation is relatively scarce. Van Os and Basten [23] formalize flexible job shop scheduling with conflict-free transportation constraints and, with a decomposition method, obtain provably optimal solutions on most benchmark instances; for a few, optimality cannot be guaranteed because of the time limit. Two-stage schemes first fix the production schedule and then resolve vehicle conflicts [20]. There is also a framework with a learning-based scheduling layer and a conflict-resolving routing layer, connected by limited feedback [7]. These works, however, address how to build a feasible, high-quality conflict-free schedule before execution starts and do not handle execution-time disturbances; for example, Sun et al. [20] assume that vehicles are always available and never fail. Conversely, the dynamic scheduling works with AGVs in Section 2.1, which do handle execution-time disturbances, do not model corridor contention between vehicles; for example, Zhang et al. [26] ignore collisions between AGVs. In other words, in research on flexible job shop scheduling with transportation, a corridor reservation table and execution-time disturbances have not yet appeared together. When both are present, the only directly applicable practice is scoping by affected operations, which yields an empty scope for a corridor blockage that does not change any operation assignment (measured in Section 6.2: 50/50). This is exactly the situation studied in this paper.

2.5 Positioning of this paper

In summary, in the flexible job shop with conflict-free AGV routing, a schedule uses operation precedence to describe processing order and time windows to record corridor occupancy. The situation described in the previous subsection then arises: a disturbance such as a corridor blockage does not act directly on machines or operations, and at the operation level neither the machine assignment nor the processing sequence needs to change; yet the corridor occupancies overlapping the disturbed interval are invalidated, so the original schedule is infeasible, and the resulting transport delays also postpone the start of downstream operations. As Table 1 shows, none of the listed works combines all three settings—operation and machine assignment, occupancy time windows, and execution-time disturbances: railway scheduling and online MAPF, which record occupancy time windows and handle execution-time disturbances, have no operation and machine assignment, whereas the works that have operation and machine assignment and handle execution-time disturbances do not record occupancy time windows and therefore do not handle corridor disturbances. With all three settings present, this paper moves the rescheduling scope onto corridor occupancies and their yielding and succession relations, which yields a reservation impact set uniquely determined by these relations.

The last columns of the table also state the principle by which this paper delineates the scope. The size of the scope puts "no misses" first: every invalidated occupancy must be included, otherwise the repaired schedule is still infeasible; subject to this, occupancies unreachable along the propagation channels are excluded and keep their original plan, but the impact set is not meant to be minimal, because a minimal set depends on the repair actions used (Remark 2). Nor are more propagation channels necessarily better; the channels must match those along which invalidation actually propagates. Omitting one class misses the occupancies affected through that class of resource— AOR expands only along jobs and machines and therefore obtains an empty scope under a corridor blockage—while including relations that do not propagate invalidation only enlarges the scope needlessly. The four channels of this paper—corridor (yielding), vehicle, job, and machine—correspond to the four classes of shared resources that constrain the timing of corridor occupancies in the model. The objects inside the scope are corridor occupancies: their routes and time intervals may be changed (and, under a vehicle failure, their vehicles as well), while the machine assignment and processing sequence of operations stay unchanged; if repair fails, the scope is enlarged in a prescribed order (Section 5).

3 The corridor-reservation setting and the execution-time recovery problem

3.1 System setting and notation

The problem studied here belongs to flexible job shop scheduling with conflict-free transportation constraints, so we build our model on the formulation of this problem by van Os and Basten [23]; the two differ in what is reserved (Section 3.1.2). This subsection introduces the objects and notation shared by the later sections. The feasible pre-disturbance schedule $\mathcal{S}^0$ is taken as a given input: the delineation in later sections depends only on $\mathcal{S}^0$ itself (machine assignment, processing sequence, and reservation table), not on the algorithm that produced it at $t = 0$, so $\mathcal{S}^0$ may come from any scheduling method.

3.1.1 Shop floor and four groups of decisions. The shop routing graph $G = (V, E)$ is strongly connected. Each machine m performs processing through a robotic arm located at a fixed node $\mathrm{loc}(m)$; a robotic-arm failure below means the failure of that machine. $v_0 \in V$ is the load/unload station. There are n jobs, a machine set M, and a homogeneous fleet of unit-load vehicles $K = \{1, \ldots, N_A\}$. Job j has a fixed chain of operations $O_{j1} \to \cdots \to O_{jn_j}$; operation O_{ji} has a candidate machine set $\Omega_{ji} \subseteq M$ and processing time t^P_{jim} on machine m. A complete schedule fixes four groups of decisions simultaneously: (i) the machine assignment x of each operation; (ii) the processing sequence π of operations

on each machine; (iii) the vehicle assignment y of each transport; (iv) a conflict-free path on G for each vehicle move. We write

$$\mathcal{S} = (x,\ \pi,\ y,\ R),$$

where the reservation table R is the explicit record of group (iv), defined below.

Transport tasks are not fixed parameters given in advance; they are determined by the machine assignment x: a transport is needed only when two consecutive operations of the same job are assigned to machines at different nodes, and when the assignment changes, the number of transports and their origin and destination nodes change accordingly. Each transport is split into two transport legs. In the *empty leg*, the vehicle travels unloaded from its current position to the node of the machine of the preceding operation to pick up the job; in the *loaded leg*, it carries the job from that node to the node of the machine of the succeeding operation. The two legs have their own task labels u and are two separate records in the reservation table. Loading and unloading times are zero: the loaded leg departs at the later of "the empty leg arrives at the pickup node" and "the job completes its preceding operation", and the delivery time is the time at which the succeeding operation can start and the vehicle can accept a new task. Later sections decide which occupancies are rewritable, which are kept as planned, and how rerouting is done with a single transport leg (empty or loaded) as the basic unit, not a whole transport or a whole operation.

3.1.2 Corridor reservation and conflict freedom. Each corridor $e \in E$ is an exclusive resource with known traversal time τ_e. Corridors are treated as single lanes: a vehicle traveling in either direction occupies the whole corridor, so at most one vehicle is on e at any time, and neither head-on encounters nor same-direction overtaking can occur. This corresponds to the single-lane narrow aisles and zone-based occupancy control common in AGV shops; a two-lane corridor can be represented as two corridors in the network. Occupancies are half-open intervals: a following vehicle may enter exactly when the preceding one leaves. Vehicles wait only at nodes and never stop inside a corridor; nodes have ample parking capacity, so waiting at nodes is unrestricted. A reservation record is

$$r = (e,\ [t^s, t^e),\ k,\ u) \in R,$$

meaning that vehicle k occupies corridor e during $[t^s, t^e)$ for leg u. Any two distinct occupancies of the same corridor must satisfy $[t^s, t^e) \cap [t^{s\prime}, t^{e\prime}) = \varnothing$. When a path is planned, each corridor is used only in time slots not yet occupied in the reservation table R, so newly written occupancies never overlap existing ones, the reservation table always satisfies the exclusivity constraint above, and no conflict check is needed afterwards.

Time-window routing is the basic module shared by our rerouting and by all baseline methods: given the origin, the destination, the earliest departure time, and the current reservation table R, it returns a path whose occupancy intervals on all corridors overlap no existing occupancy, together with the entry time of each corridor. We implement it as a time-window Dijkstra algorithm [20]. We do not modify this module; we change only *which* transport legs are handed to it for replanning. Van Os et al. reserve nodes [23], and Lyu et al. additionally impose node-capacity constraints [9]; we reserve only corridors (the edges of the network) and place no reservations on nodes, so our results cannot be compared directly with the values reported in these two works (Section 7.1).

3.1.3 Operation-precedence graph and reservation dependency graph. The operation-precedence graph $G_T = (T_T, E_T)$ (also called the task graph in the literature; the subscript T stands for task) has operations as vertices; its directed edges represent precedence constraints between two operations and are of two kinds: *job precedence*, from operation O_{ji} of a job to its next operation $O_{j,i+1}$; and *same-machine precedence*, from an operation on a machine to the operation

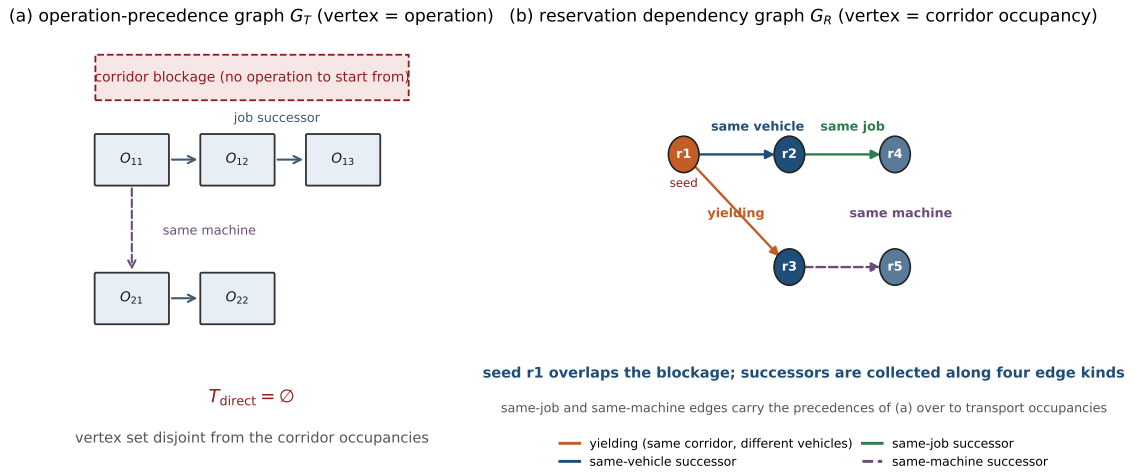


Fig. 2. The vertex sets of the two graphs are disjoint (schematic construction). (a) Operation-precedence graph G_T: vertices are operations, and a corridor blockage finds no seed operation. (b) Reservation dependency graph G_R: vertices are corridor occupancies; starting from the occupancies overlapping the blockage window, subsequent occupancies are included along the four kinds of dependency edges (yielding, same vehicle, same job, same machine; Definition 4).

processed immediately after it on that machine. G_T contains no corridor occupancies and therefore does *not* contain the yielding relations between them. The reservation dependency graph $G_R = (R, E_R)$ has the corridor occupancies in the reservation table R as vertices; a directed edge $r \longrightarrow r'$ means that invalidating r may force r' to be rerouted, and there are four kinds (Definition 4): *yield* edges between two consecutive occupancies of the same corridor, *same-vehicle* edges between two adjacent occupancies of the same vehicle, and *same-job* and *same-machine* edges, obtained by carrying the job precedence and same-machine precedence of G_T over to transport occupancies. Here the *transport occupancies* of an operation are the corridor occupancies produced by the transport legs that move the job for that operation. The vertex sets of the two graphs are disjoint: one represents processing, the other traversal. Hence there is a class of disturbances that changes only when corridors are passable and no operation parameter (e.g., a corridor blockage): scoping on G_T finds no seed operation, whereas on G_R, as long as some planned corridor occupancy overlaps the disturbed interval (exactly the case in which the original schedule becomes infeasible), these occupancies form a nonempty seed set from which delineation can start. Figure 2 contrasts the two graphs, Figure 1 shows the consequences under a corridor blockage, and Table 2 summarizes the notation used in this section and later.

3.2 Execution-time assumptions

The execution-time conventions and physical setting are as follows. A1–A4 are shared by all compared methods, so that differences in results can be attributed to how the rescheduling scope is delineated rather than to model differences. A5 constrains only our method; the global-rescheduling baseline may additionally change the machine assignment and the processing sequence (see the last sentence of A5):

A1. Feasible initial schedule. $\mathcal{S}^0$ passes the conflict-freedom check before the disturbance, and the reservation table is consistent with the operation timetable.

Table 2. Frequently used notation

Symbol	Meaning
$G = (V, E)$, τ_e	Corridor network; traversal time of corridor e
v_0, $\mathrm{loc}(m)$	Load/unload station; node of robotic arm m
x, π, y	Machine assignment, processing sequence, vehicle assignment
R, $r = (e, [t^s, t^e), k, u)$	Reservation table; a half-open occupancy (vehicle k, leg label u)
G_T / G_R	Operation-precedence graph / reservation dependency graph
$\mathcal{S}^0$, $\mathcal{S}'$	Schedule before the disturbance, schedule after recovery
$\mathcal{D}$, t_{now}	Disturbance event; occurrence (decision) time
T_{direct}, T_{impact}	Directly disturbed operations on the operation-precedence graph / θ-step scope
$\mathrm{Src}(\mathcal{D})$, U	Seed occupancies; set of rewritable occupancies
$\mathrm{Cl}(\cdot)$	Reservation impact set (obtained by STRC)
φ	Fraction of disturbed future operations (magnitude sweep)

A2. No backtracking on finished occupancies. Only unfinished occupancies with $t^e > t_{\mathrm{now}}$ and unfinished operations are rewritten; the fields of occupancies with $t^e \le t_{\mathrm{now}}$ are not rewritable.

A3. Single, fully observed disturbance. The disturbance is fully observed at t_{now} and does not overlap with other disturbances. We do not study the following cases: several disturbances arriving one after another; disturbance information (e.g., the blockage duration) that can only be estimated, possibly wrongly, when the disturbance occurs; and rescheduling repeatedly as new information arrives.

A4. Exclusive corridors, ample buffers, homogeneous unit-load fleet. Corridors remain exclusive half-open resources, and a blockage is injected into the routing layer as an additional occupancy, i.e., an interval during which the corridor may not be entered, called the *blockage window*. Finite buffers, charging, acceleration and deceleration, and loading/unloading durations are not modeled; all vehicles start empty at v_0 at the beginning of the plan.

A5. Permitted recovery actions. For Class B disturbances, the original machine assignment x and processing sequence π are kept and only rerouting is performed; upon failure, the set of rewritable occupancies is escalated. Class A failures additionally allow the following: a vehicle failure removes the failed vehicle from the fleet and reassigns its tasks to another vehicle; a robotic-arm failure reassigns unfinished operations to other feasible machines. The processing sequence π is never changed, and the search is not restarted. The global-rescheduling baseline in the experiments may change x and π (Section 5.3).

3.3 A dichotomy of disturbances

Definition 1 (Class A/B disturbances). *A disturbance is of **Class A** if it directly invalidates the assignment or* executability *of some operations (robotic-arm failure, significant processing-time deviation, rush-order insertion, vehicle failure, etc.). It is of **Class B** if it only makes some corridors unavailable for some period without making any operation inexecutable (corridor blockage, corridor slowdown, etc.).*

Remark 1 (Why vehicle failure is Class A). *A vehicle failure does not change the machine assignment of any operation, so at first sight it seems to belong with corridor blockages. However, the criterion in Definition 1 is* executability, *not assignment: after a vehicle fails, the operations it transports cannot start under the original plan. As long as the method records which operations each vehicle transports, these operations can serve as seeds on the operation-precedence graph, and operation-level scoping gives a nonempty scope; vehicle failure is therefore of Class A. If the method records only operations*

and machines (as common AOR implementations do), a vehicle failure also yields an empty scope; the baseline in Section 6.8 records this correspondence.

Definition 2 (Direct set and affected set on the operation-precedence graph). *For a disturbance $\mathcal{D}$, let $T_{\text{direct}}(\mathcal{D})$ be the set of operations that cannot be executed as planned because of a robotic-arm failure, a vehicle failure, or operation invalidation. For Class B corridor disturbances, we set $T_{\text{direct}} = \varnothing$ by convention. Starting from T_{direct} on G_T, the operations reachable within at most θ steps along job-successor and same-machine-successor edges (including T_{direct} itself) form $T_{\text{impact}}(\mathcal{D}, \theta)$.*

Definition 3 (Seed occupancies). *The seed set $\mathrm{Src}(\mathcal{D}) \subseteq R$ is given by disturbance type. Every type includes only occupancies with $r.t^e > t_{\text{now}}$, i.e., occupancies not yet finished at the decision time; finished occupancies are never seeds.*

(a) ***Corridor blockage/slowdown*** *$\mathcal{D} = (e, [t_a, t_b))$: $\{ r : r.e = e,\ [r.t^s, r.t^e) \cap [t_a, t_b) \neq \varnothing \}$, i.e., the occupancies of corridor e that overlap the disturbed interval. For multiple short blockages, the seed set is the union of the seed occupancies of the individual blockage intervals. Slowdown and blockage share this seed rule: the original schedule was written with the unslowed traversal times, so an occupancy overlapping the disturbed interval cannot finish at its original time under either disturbance, and the two yield the same reservation impact set. They differ only in rerouting: during a blockage the corridor is impassable; during a slowdown it remains passable, but the part of the traversal time that falls inside the disturbed interval is lengthened by a factor.*

(b) ***Vehicle failure*** *$\mathcal{D} = (k)$: all occupancies of that vehicle after t_{now}, $\{ r : r.k = k \}$.*

(c) ***Robotic-arm failure*** *$\mathcal{D} = (m, F)$, where F is the set of unfinished operations on that machine: for each job, take the smallest operation index $i_{\min}(j)$ in F; the seed occupancies are all unfinished occupancies of each job from that operation onward (inclusive). We take the job chain forward, rather than only the occupancies of the operation itself, because the inbound transport of that operation may have finished before t_{now}: taking only the operation itself would leave the seeds empty or too few, so that the reservation impact set would cover fewer operations than the affected set on the operation-precedence graph, and the comparison of the two scopings would be unfair—the difference would come from how seeds are taken, not from the scoping object.*

Proposition 1 (The operation-precedence graph yields an empty scope for Class B). *Let $\mathcal{D}$ be a corridor blockage. Under the Class B convention of Definition 2, $T_{\text{direct}}(\mathcal{D}) = \varnothing$, and hence $T_{\text{impact}}(\mathcal{D}, \theta) = \varnothing$ for every finite $\theta \geq 0$. If, in addition, $\mathrm{Src}(\mathcal{D}) \neq \varnothing$, then $\mathcal{S}^0$ is infeasible under $\mathcal{D}$.*

Proof. A corridor blockage only makes a corridor unavailable for some period without making any operation inexecutable, so it is of Class B by Definition 1. This step relies on two premises: the blockage window $[t_a, t_b)$ is finite, and by Assumption A4 buffers are ample, so vehicles can wait at nodes until the blockage ends and then traverse. Hence, even if the network is disconnected during the blockage, no operation becomes inexecutable; operations are only delayed. If the blockage were indefinite, or buffers were insufficient to support waiting, the disturbance would be of Class A by Definition 1 and would fall outside the scope of this proposition. Being of Class B, it has $T_{\text{direct}} = \varnothing$ by the Class B convention of Definition 2. The starting set of the θ-step expansion is empty, so $T_{\text{impact}} = \varnothing$. If some $r \in \mathrm{Src}(\mathcal{D})$ exists, the occupancy interval of r intersects the blockage window. Interpreting $\mathcal{D}$ as a mandatory occupancy of that corridor, r and the blockage window overlap in time on the same exclusive resource, which violates the corridor exclusivity constraint of Assumption A4. Hence $\mathcal{S}^0$ is infeasible. □

Proposition 1 is a counterexample at the problem level. Its first part follows directly from the Class B convention of Definition 2 (by definition, the operation-precedence graph does not contain the yielding relations between corridor occupancies and thus cannot represent a corridor blockage). The nontrivial part is the second: when the seed set is nonempty, the original schedule is already infeasible. Hence "an empty scope on the operation-precedence graph ⇒ no rescheduling needed" does not hold in a setting where corridors are reserved by time windows. The counterexample does not depend on any repair algorithm; Section 6.2 verifies this case on each of the 50/50 pairs.

3.4 The recovery problem

Problem 1 (Bounded rescheduling with the scope delineated on the reservation table). *Given a feasible schedule $\mathcal{S}^0$, a disturbance $\mathcal{D}$, and a time t_{now}, find $\mathcal{S}'$ such that (i) $\mathcal{S}'$ passes the conflict-freedom check under $\mathcal{D}$; (ii) occupancies outside the rewrite scope keep their corridors, time intervals, and vehicles as close to $\mathcal{S}^0$ as possible; (iii) the runtime is subject to a budget. The secondary objective is the makespan $C_{\max}(\mathcal{S}')$.*

Bounded rescheduling proceeds in two steps: first delineate the set of rewritable occupancies $U \subseteq R$; then reroute only the transports in U, keeping occupancies outside the set as planned; we call the second step *bounded repair* (Section 5). We obtain U as Cl(Src); how the baselines obtain U is described in Section 4.

4 The spatiotemporal reservation closure

This section determines the rewritable occupancy set U. Given the reservation table R and the seed set $\text{Src}(\mathcal{D})$ (Definition 3), we build a dependency graph over occupancies and take all occupancies reachable from the seeds along its edges, i.e., $U = \text{Cl}(\text{Src})$. How vehicles are rerouted inside U is described in Section 5.

4.1 Reservation dependency edges

Definition 4 (Reservation dependency graph). *The vertices are the corridor occupancies in R. A directed edge $r \longrightarrow r'$ means: once r is rewritten, r' may have to be rewritten as well. The edges are of four kinds:*

(1) ***Yield edges:*** *two occupancies of the same corridor by different vehicles whose time intervals overlap or meet end to start ($r.t^e = r'.t^s$); the edge points from the one with the earlier entry time t^s to the one with the later entry time;*

(2) ***Same-vehicle successors:*** *two occupancies of the same vehicle that are adjacent in time, from the earlier to the later;*

(3) ***Same-job successors:*** *from the transport occupancies of operation i of a job to the transport occupancies of operation $i+1$;*

(4) ***Same-machine successors:*** *for two operations processed consecutively on the same machine, from the transport occupancies of the former to those of the latter.*

This yields $G_R = (R, E_R)$.

Figure 2(b) illustrates the four kinds of edges. Two occupancies that meet end to start do not conflict under exclusive semantics, but once the former is delayed the latter must give way, so they are also joined by a yield edge. The edge set is a modeling choice: it may include occupancies that need not be rewritten, and it may miss dependencies that are not modeled (Remark 2).

Definition 5 (Reservation impact set and spatiotemporal reservation closure). *Given a seed set $S \subseteq R$ and a time t_{now}, denote the occupancies unfinished at the decision time by*

$$R^\circ = \left\{ r \in R : \; r.t^e > t_{\text{now}} \right\},$$

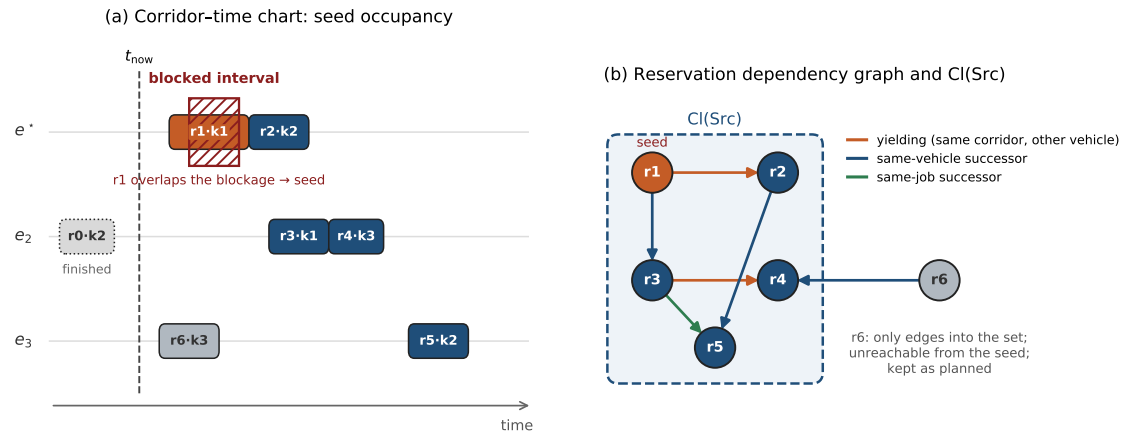


Fig. 3. Construction of the reservation impact set (the same occupancies appear in both panels). (a) Corridor–time chart: each block is a corridor occupancy, labeled "occupancy·vehicle"; only r_1 overlaps the blocked interval of corridor $e^\star$ and becomes the seed; r_0 finished before t_{now} and takes no part in delineation. (b) Dependency edges among these occupancies (colors distinguish the kinds): r_2–r_5 are reachable from r_1 along the edges and, together with r_1, form the reservation impact set Cl(Src) (dashed box). r_6 has only an edge into the set, is unreachable from the seed, stays outside, and is kept as planned; no edge leaves the set.

and let G_R° be the subgraph that keeps only the occupancies in R° and the edges among them. The reservation impact set is defined as

$$\mathrm{Cl}(S) = \left\{ v \in R^\circ : \exists\, s \in S \cap R^\circ \text{ such that } s \to^* v \text{ in } G_R^\circ \right\},$$

that is, all unfinished occupancies reachable along edges from the unfinished seeds (the seeds themselves included). The procedure that computes $\mathrm{Cl}(S)$ is called the spatiotemporal reservation closure (STRC).

With G_R° stored as adjacency lists, one breadth-first search from the seeds yields $\mathrm{Cl}(S)$ in $O(|R^\circ| + |E_R^\circ|)$ time, where E_R° is the edge set of G_R°. The experiments do not time this step separately; it is included in the total runtime reported in Section 6.9.

Figure 3 illustrates this construction on a small example.

The baseline that scopes on the operation-precedence graph takes U to be the transport occupancies of the operations in T_{impact} that are unfinished at t_{now}; by Proposition 1, this set is empty under Class B disturbances. Our method takes $U = \mathrm{Cl}(\mathrm{Src})$. In the experiments the two are denoted R1 and R2, respectively.

4.2 Closure

Proposition 2 (Structural closure). *For any $S \subseteq R$, G_R° contains no edge $u \to v$ with $u \in \mathrm{Cl}(S)$ and $v \in R^\circ \setminus \mathrm{Cl}(S)$. Consequently, no unfinished occupancy outside the impact set is reachable along edges from $\mathrm{Cl}(S)$.*

Proof. Suppose such an edge $u \to v$ exists. Since $u \in \mathrm{Cl}(S)$, there is an unfinished seed s with $s \to^* u$. Appending $u \to v$ gives $s \to^* v$, so $v \in \mathrm{Cl}(S)$ by Definition 5, a contradiction. □

Proposition 2 is a direct corollary of the definition: once the edge set is fixed, the impact set is closed under these edges. Whether it covers all dependencies that actually exist physically depends on the choice of edges; this is discussed in the following remark.

Remark 2 (The reservation impact set and the minimal rewritable set). *Closure states only that no dependency edge leads from inside the set to outside it; it neither guarantees that the impact set suffices to restore feasibility nor shows that it is small enough. We discuss the relation between the impact set and the minimal rewritable set in three points.*

First, ***the minimal rewritable set depends on the repair procedure used, whereas the reservation impact set does not****. To call a set U minimal, one must first specify "under which repair capability U suffices to restore feasibility": the minimal U when reassignment and resequencing are allowed differs from the minimal U when only rerouting is allowed. The reservation impact set is uniquely determined by G_R and independent of the repair procedure, so it cannot, and is not meant to, coincide with the minimal set of any particular repair procedure.*

Second, ***the four kinds of edges include occupancies that need not be rewritten; the known case of missed occupancies (meeting end to start) has been folded into the yield edges****. A yield edge is created whenever two occupancies overlap, without checking whether the latter* truly *can proceed only by waiting for the former—if a bypass exists there, the latter can simply detour and need not be rewritten; likewise, same-vehicle successor edges may pull in chain tails that the delay cannot reach. Both effects make the impact set larger than necessary. To test the treatment of occupancies that meet end to start, we run a* difference probe*: on the same* 50 *instance pairs as Section 6.4, each occupancy in the impact set is delayed alone by* 1 *time unit, all else unchanged, and we check whether it newly overlaps an occupancy outside the impact set on the same corridor. Each such overlap counts as a leak, i.e., a dependency that the current edge set does not draw but that would cause a conflict after the delay. The total number of leaks is 0, and the number of pairs with a leak is 0/50; in 44 of the pairs the impact set leaves some occupancies outside, and in the rest it already contains all unfinished occupancies. This test targets only the known omission of occupancies meeting end to start and does not prove the edge set complete.*

Third, ***empirically, the impact set is not small****. Under corridor blockage, the reservation impact set covers on average about 92% of the occupancies unfinished after t_{now} (Section 6.8); compared with rewriting all unfinished occupancies, it leaves out less than one tenth. Bounded repair is fast mainly because it does not restart the search, not because far fewer occupancies are to be rewritten; "local" and "bounded" do not mean that the changes are small. This difference of less than one tenth still lowers the rewrite fraction by 0.064 on average, while runtime and makespan remain nearly unchanged (Section 6.9).*

What this paper provides, therefore, is a rewritable set that is uniquely determined by structure, reproducible, and closed under the dependency edges, and that sufficed to restore feasibility in all our experiments; approximating the minimal rewritable set is a separate problem, left for future work.

4.3 Can the impact-set size be predicted from the network structure?

The size of the reservation impact set is uniquely determined by G_R: once the schedule and network are given, there is no tunable parameter. When the instances are scaled from $8 \times 4 \times 4$ up to $20 \times 10 \times 10$, the share of unfinished occupancies covered by the impact set stays between 0.78 and 0.91 and does not rise with size (Section 6.11). Hence the extent of "local" is measurable, but its size is not small (Remark 2).

The impact-set size is determined jointly by the schedule and the network. The experiments examine indicators computed from the network alone and independent of the schedule (the load/unload minimum cut and the funnel share)

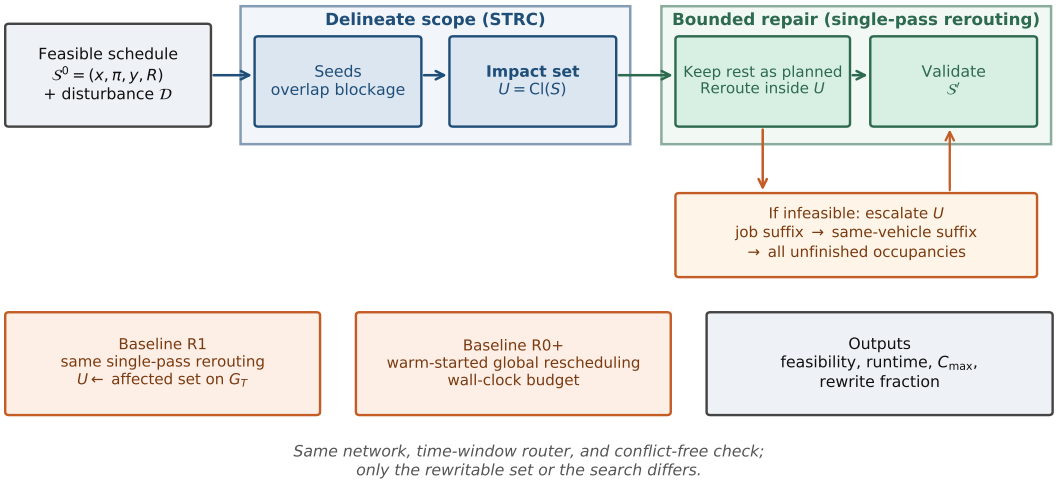


Fig. 4. Bounded repair scoped on the reservation impact set. Top row: the impact set is obtained from the seeds and vehicles are rerouted; middle row: the rewritable scope is escalated after failure; bottom row: the baseline scoping on the operation-precedence graph, the warm-started global rescheduling baseline, and the output metrics. Three further baselines with millisecond runtimes (RS, RD, RA) are not drawn; see Section 6.9.

and find that they do not predict the impact-set size: on external networks these indicators are constant, whereas the impact-set share changes markedly when the machine configuration on the same network is changed (Section 6.6).

5 Bounded repair on the reservation impact set

Figure 4 outlines the procedure of this section. We first delineate the rewritable occupancy set U by the reservation impact set, then replan paths and intervals only for the occupancies in U, keeping occupancies outside it as planned; if the result is still infeasible, we escalate U and retry. Section 5.1 gives the rerouting procedure and its guarantee, Section 5.2 gives the rule for escalating the scope after failure, and Section 5.3 describes the baselines. All methods share the same corridor network, time-window routing, and conflict-free check.

5.1 Single-pass rerouting that keeps the rest as planned

Given the rewritable occupancy set U, single-pass rerouting keeps the original vehicle and machine assignments and reschedules the transports in U one by one in the original processing order. Rerouting works per occupancy: the part of a trip that a vehicle has already traversed before t_{now} consists of finished occupancies, which are pre-committed unchanged, and rerouting plans only from the node where the vehicle is at that time. Algorithm 1 gives this basic operation; Algorithm 2 (Section 5.2) wraps it, computing the reservation impact set and escalating U on failure.

By complexity, delineating the scope needs only one breadth-first search (Definition 5), whereas one call of ReplayReroute invokes a time-window shortest path once for each transport leg in U; the runtime comes mainly from the latter. Rerouting launches no population-based search. The comparison in Section 6.4 fixes Algorithm 1 and replaces

Algorithm 1 Single-pass rerouting ReplayReroute

Require: schedule $\mathcal{S}^0 = (x, \pi, y, R)$, disturbance $\mathcal{D}$, rewritable set U, time t_{now}
Ensure: schedule $\mathcal{S}'$ or failure

write $\mathcal{D}$ into the routing layer ▷ blockage: write the blockage window; slowdown: traversal time inside the window lengthened by a factor
take an empty table R'
for each occupancy not in U (including all finished occupancies) **do**
    pre-commit the original fields $(e, [t^s, t^e), k)$ to R' ▷ finished occupancies must be written unchanged; omitting them amounts to rewriting them
    if conflict with the blockage window **then**
        return failure ▷ the seeds do not cover an occupancy overlapping the blockage window
    end if
end for
for each $r \in U$, in the original processing order **do**
    plan from the node where the vehicle is at t_{now} ▷ the traversed part was pre-committed in the previous step and is not committed again
    invoke time-window routing on the current R' and commit the new occupancies
    update the idle time of the machine and the ready time of the job concerned according to π
end for
independent check: no conflicts, no occupancy inside a blockage window, traversal times inside slowdown windows lengthened, fields outside the set unchanged
if the check passes **then**
    return $\mathcal{S}' = (x, \pi, y, R')$
else
    return failure
end if

only the source of U (the operation-precedence graph or the reservation impact set), so that a difference in feasibility rate between the two scopings can come only from the scope object.

Proposition 3 (A sufficient condition for unchanged fields outside the set). *Let the rewritable occupancy set be $U = \mathrm{Cl}(S)$, let single-pass rerouting be executed by Algorithm 1, and suppose: (i) the blockage windows have been written into the routing layer; (ii) for every $r \in R^\circ \setminus U$, its original fields $(e, [t^s, t^e), k)$ have been pre-committed to the new reservation table, and the pre-commit succeeds; (iii) the time-window router replans only the occupancies in U; (iv) the final schedule passes the conflict-free check. Then, in the recovered reservation table R', the corridor, interval, and vehicle of every $r \in R^\circ \setminus U$ agree with $\mathcal{S}^0$. The proposition holds per occupancy, not per whole transport: a transport may have some occupancies in U and others outside it, in which case only the part outside U is protected by this proposition.*

Proof. By (ii), every occupancy outside the set is written into the new table with its original interval before the replay begins, and it does not conflict with the blockage windows (otherwise the pre-commit would fail). By (iii), the replay commits new occupancies only for occupancies in U and neither deletes nor rewrites the committed occupancies outside the set. Hence every occupancy outside the set has the same $(e, [t^s, t^e), k)$ in the new table as its pre-committed value, i.e., as in $\mathcal{S}^0$. The check in (iv) only rules out conflicts between new and existing occupancies; it does not alter committed fields. □

Premises (i)–(iv) are execution conditions of the rerouting procedure itself and can be checked one by one in the implementation; beyond them, the proposition relies on A1 (the initial schedule is feasible; otherwise the original fields

outside the set might conflict with one another and the pre-commit would necessarily fail), A2 (finished occupancies are not revisited), and A4 (corridors are exclusive).

Remark 3 (When the premises of Proposition 3 fail). *There are three cases. First, a failed pre-commit: if an occupancy outside the set overlaps a blockage window, premise (ii) fails, which means that the seed set S does not cover all occupancies overlapping the blocked interval, i.e., the seed set is incomplete. Second, scope escalation: if rerouting fails and triggers the scope escalation of Section 5.2, the rewritable set becomes $U' \supsetneq U$, and the proposition holds only for the occupancies that finally remain outside the set. Escalation gives priority to feasibility rather than to minimal rewriting; escalations are reported separately in the experiments. Third, too coarse an implementation granularity: premise (iii) requires the time-window router to replan* only *the occupancies in U. If an implementation decides whether to reroute at a unit coarser than an occupancy (e.g., a whole transport or a whole operation), occupancies outside U are rewritten along with it, and the conclusion of the proposition fails. The implementation must therefore decide rerouting per occupancy. The granularity must in fact go down to segments: for a transport leg spanning t_{now}, the segments already traversed are kept as they are, and rerouting starts from the node where the vehicle is at t_{now}; otherwise finished occupancies are rewritten, violating Assumption A2. Because these segments lie inside U, a comparison of the fields outside the set cannot detect such rewrites.*

5.2 Scope escalation after failure

Escalating the scope step by step after failure has precedents. Local Rescheduling [6] relaxes the time window step by step until feasibility is reached; AOR adds subsequent operations along same-job and same-machine successors (Section 2.1). We escalate U along reservation dependency edges rather than along time windows. This step ensures that feasibility can still be restored when the impact set is too tight, without switching immediately to a global search. E3 disables this step in order to compare the feasibility rates of the two scopings (Section 6.4).

With all occupancies outside U fixed, if the occupancies in U find no feasible path and interval (e.g., a transport cannot arrive when its operation requires), Algorithm 1 returns failure. We then escalate U step by step in the following order and retry the same rerouting procedure; this is called *scope escalation* below: (i) $U \leftarrow U \cup \{$the unfinished occupancies in the same-job suffixes (all later transport occupancies of the job) of the operations currently in $U\}$; (ii) $U \leftarrow U \cup \{$the unfinished occupancies in the same-vehicle suffixes (all later occupancies of the vehicle) of the vehicles owning occupancies in $U\}$; (iii) $U \leftarrow \{\, r \in R :\ r.t^{e} > t_{\mathrm{now}} \,\}$, i.e., every occupancy unfinished at the decision time becomes rewritable; this is exactly the rewritable set of the baseline RA (Section 5.3). After escalation, rerouting remains a deterministic single pass. In the experiments with escalation enabled (the disturbance-magnitude sweep and E8), feasibility was restored in all instance–seed pairs, with runtime staying in the range of milliseconds to a little over ten milliseconds (Sections 6.10 and 6.11).

Algorithm 2 combines Section 5.1 and this section, corresponding to Figure 4.

5.3 Baselines

The global rescheduling baseline (denoted R0+) searches anew under the same no-entry constraints and within a time limit. R0+ is a genetic algorithm that encodes (x, π) as a chromosome. One *decoding* means: following the chromosome's assignments and processing order, invoke the time-window routing of Section 3.1.2 from scratch for all transports on the reservation table into which the blockage windows have been written, obtaining a new R and $C_{\max}$. R0+ decodes repeatedly and may modify the chromosome. Its decoding does not rewrite finished occupancies: these are

Algorithm 2 Bounded repair on the reservation impact set (single-pass rerouting + scope escalation)

Require: schedule $\mathcal{S}^0$, disturbance $\mathcal{D}$, time t_{now}
Ensure: schedule $\mathcal{S}'$ or failure
1: $S \leftarrow \text{Src}(\mathcal{D})$; $U \leftarrow \text{Cl}(S)$
2: **for** round = 0, 1, 2, 3 **do**
3: $\quad \mathcal{S}' \leftarrow \textsc{ReplayReroute}(\mathcal{S}^0, \mathcal{D}, U, t_{\text{now}})$
4: $\quad$ **if** $\mathcal{S}'$ is feasible **then**
5: $\quad\quad$ **return** $\mathcal{S}'$
6: $\quad$ **end if**
7: $\quad$ update U by the scope-escalation rule ▷ job suffix / same-vehicle suffix / all unfinished
8: **end for**
9: **return** failure

pre-committed, and only operations not yet started are scheduled from t_{now} on (Section 6.7). R0+ is warm-started, i.e., the original schedule is placed in the initial population as one individual, so as not to discard the assignment and path information already in it. Because R0+ can reassign and resequence, it is more likely to shorten $C_{\max}$, but its runtime is set by the budget; the two methods are compared in makespan and runtime in Section 6.7.

Comparing only with R0+ leaves one question open: within a millisecond runtime, is scoping on the impact set necessary? To answer it, we implement three approaches that do not restrict the scope by the impact set. Global right shift (RS) does not determine which occupancies are affected; it simply pushes all unfinished occupancies past the end of the blockage. Re-decoding of the original chromosome (RD) leaves the chromosome unchanged: the finished part is kept, and the rest is decoded once from the original chromosome. **RA** makes every occupancy still unfinished after t_{now} rewritable and then runs *exactly the same* rerouting procedure as in Section 5.1. It therefore differs from our method only in "which occupancies are rewritable" and is the only baseline that isolates the effect of the reservation impact set. All three run within milliseconds and thus serve as baselines for bounded repair *within the same runtime range*; their definitions and results are given in Section 6.9.

6 Experiments

6.1 Experimental setup

Network, routing, and instances. The lower-level network, the time-window router, and the independent checker are fixed. The extended experiment matrix uses **five instances**, with size written as "jobs × machines × AGVs": a small $3 \times 3 \times 2$ instance for step-by-step checking, a congested $8 \times 4 \times 4$ instance, and three $8 \times 4 \times 4$ spine layouts, funnel, high, and mid. The first two are hand-crafted; the last three are produced by our instance generator, whose parameters include the congestion level, heterogeneity h, and flexibility f. In a spine layout, a main corridor (the spine) runs through the machine area and each machine hangs on one of two hubs of the spine via its own branch; the load/unload station sits at one end of the spine and is connected by one or two parallel exits [19]. Every transport must pass the same exit and the same middle section, so contention concentrates on these few corridors; the congested instance also has this layout. The three spine layouts have the same size, heterogeneity, and flexibility and differ only in the number of parallel lanes: funnel has one lane at the station exit and one in the middle section, high doubles the exit, and mid additionally doubles the middle section. Adding or removing a lane changes only how many vehicles can pass at once, not the travel distance, so differences in impact-set size among the three can be attributed to lane capacity rather than to problem size. The random seed used to generate an instance determines only the instance itself and is independent

of the run seeds below, which determine the disturbance injection and the search process. **We use ten run seeds**, all fixed once before the experiments and not selected by results: 42, 7, 2024, 99, 123, 13, 1, 777, 31415, and 8; E5 uses the first three of them. The five instances and ten seeds give $5 \times 10 = 50$ instance–seed pairs ("pairs" for short below). All per-pair comparisons, win/loss counts, and paired tests in this paper use the instance–seed pair as the unit: in the same pair, all methods read the same initial solution, the same blockage, and the same decision time. The initial schedule is obtained by replaying the time-window router of Section 3.1.2 in processing order, and it passes the independent conflict-free check; all baseline methods share this initial solution within the same instance–seed pair. We require only that $\mathcal{S}^0$ be feasible before the disturbance.

Implementation and computing environment. All experiments are implemented in pure Python with the standard library only, without third-party numerical libraries; the lower-level network, the time-window router, and the independent checker reuse the same implementation as our static integrated work. Everything runs on a single machine in a single thread, without parallelism: Intel Xeon W-1270 (8 cores, 16 threads), 64 GB RAM, Windows 10, and the CPython 3.7.6 interpreter (64-bit). All timing results were obtained under the single-machine, single-thread setting above. Different hardware would shift them uniformly, so all our runtime conclusions are stated as *ratios* or *orders of magnitude* and do not rely on absolute milliseconds.

Second batch: layouts from the published literature. The five instances above use layouts we constructed; the three spine layouts have the same node positions and route lengths and differ only in the number of parallel lanes, i.e., in how many vehicles can pass at once. To test whether the conclusions depend on these particular layouts, we run an additional batch of 5 instances. The grid size, the nodes of the load/unload station and of each machine, and which adjacent node pairs of the grid have no corridor between them are transcribed item by item from the layout figures in Appendix A of Lyu et al. [9], and the transcription is checked item by item by a script against the machine-readable encoding shipped with the van Os toolkit. The number of machines varies with the layout, from 4 to 8. The same ten random seeds are used, giving 50 instance–seed pairs in total. The two batches share the number of jobs, the number of AGVs, the number of operations per job, h, f, and the calibration target of the transport-to-processing time ratio, so any difference between the two batches can come only from the layouts themselves.

That appendix gives six layouts in order (called layouts 1–6 below), with 3 to 8 machines respectively. We fix the flexibility at $f = 0.6$ for all instances; with only 3 machines, each operation then has on average only 1.8 candidate machines, short of the requirement that "every operation has at least two candidate machines". Relaxing f for this single layout would introduce an extra variable, so layout 1 is not used.

The remaining five layouts **fall on only 2 distinct networks**. Layouts 2 and 3 (4 and 5 machines) share the same 4×4 grid: 16 nodes, where adjacent nodes would give 24 corridors, but in the layout figure one pair of adjacent nodes has no corridor between them, leaving 23. Layouts 4–6 (6, 7, and 8 machines) share the same 5×5 grid: 25 nodes, 40 corridors in a full grid, two of which are absent, leaving 38. The five layouts differ in the number and positions of machines, not in network shape. Therefore, when Section 6.6 analyzes this batch, **the number of independent samples along the network-shape dimension is 2, not 5**, whereas along the machine-configuration dimension (number and positions) it is 5; all claims below are stated according to this division.

This batch differs from the data published by Lyu et al. in three respects. **First, the per-edge travel times are filled in by us.** Lyu et al. give per-edge durations only for a single example instance (and they are not uniform); the Appendix A layouts come without per-edge durations. We fill them in with "all edges equally weighted," which is consistent with the "constant unit-step duration" assumption of the van Os model and differs only in the scale of the time unit. Consequently, this batch is not compared numerically with the values reported by Lyu or van Os; we borrow

only their networks. **Second, the loading and unloading stations are merged into one load/unload station.** The original layouts have separate loading and unloading stations; we follow the single load/unload station setting common in the joint machine–AGV scheduling literature [2] (Section 3), in which jobs depart from the same station v_0 and return to it after completion. We therefore take the loading station at the vehicles' starting location as v_0 and keep the original unloading station as an ordinary grid node. The network itself (grid, adjacent node pairs without a corridor, and the node of each machine) is unchanged; only the endpoints of each job's first and last transport change. **Third, their job data are not used.** In the public instance families we have measured, the average transport time is only a few percent of the average processing time, so transport hardly causes any contention; if their job data were also adopted, corridor contention and all phenomena that depend on it would become unobservable. Hence this batch takes only their networks, and the job data are generated with the same parameters as the first batch.

The van Os toolkit also contains a set of layouts labeled Liu 2023, which is not included in this batch. These layouts allow vehicles to move diagonally, whereas our network contains only corridors between nodes adjacent up, down, left, or right (Section 3); the two network models differ, so these layouts are not used.

Disturbances. All experiments use the same decision time $t_{\text{now}} = 0.35\, C_{\max}(\mathcal{S}^0)$. This value was fixed before the experiments: at this time about 40% of the occupancies have finished and 60% are still unfinished, so there is both an executed part that must stay unchanged and a sufficiently large part to repair; we did not perform a sensitivity analysis on this value. The disturbances are as follows. E1–E3, E5, and E7 block one busy corridor, namely the corridor with the most occupancies still unfinished at t_{now}; the blockage is one continuous time window that runs from the earliest start to the latest end of these occupancies and therefore covers all of them. We call this disturbance a *busy-corridor blockage* below. E4 blocks the corridors in the load/unload minimum cut (Section 6.6). E6 adds three further disturbance types at the same decision time (Section 6.8). The disturbance-magnitude sweep first sorts the operations that still have unfinished occupancies at t_{now} by the start time of their earliest such occupancy and takes the first fraction φ as disturbed operations; each unfinished occupancy of these operations then generates its own blockage, whose corridor and interval are exactly those of the occupancy. The blockages are not merged into one long window, since a merged window would also cover the occupancies of unselected operations lying in between.

Baseline methods. Six methods are compared, listed in increasing order of their difference from our method:

- **R2 (our method, STRC)**: scoping on the reservation impact set, followed by single-pass rerouting. Scope escalation is enabled only in the magnitude sweep and in E8, and disabled in E1–E3, E5, and E7.
- **R1**: scoping on the affected set of the operation-precedence graph, followed by the same single-pass rerouting as R2, with $\theta = 2$ throughout. θ is the maximum number of hops of forward expansion along the within-job operation precedence on the operation-precedence graph; it affects only $|U_{R1}|$ under Class A disturbances. A Class B disturbance acts directly on no operation, so the seed operation set is empty and the result is empty however many hops are expanded (Proposition 1). R1 and R2 differ only in the scope object.
- **RA**: no restriction to the impact set; all occupancies still unfinished after t_{now} are made rewritable, and the rerouting procedure is identical to R2, so that the role of the reservation impact set is tested in isolation (Section 6.9).
- **RS**: global right shift. It neither scopes nor reroutes and simply pushes all unfinished occupancies past the end of the blockage (Section 6.9).
- **RD**: re-decoding of the original chromosome. The chromosome is unchanged and decoded once with a fixed prefix; this is the cheapest global recomputation in our implementation (Section 6.9).

- **R0+**: global rescheduling that searches again within a time limit, starting from the original schedule. It is a genetic algorithm with population size 40, with the chromosome of the original schedule seeded into the initial population; it uses the same fixed-prefix decoding as RD, so occupancies before t_{now} are not rewritable. *No generation limit or early-stopping threshold is set*; it terminates only on the wall-clock budget, whose values are given in Section 6.7.

The five baselines differ from R2 by successively larger steps: R1 only in the scope *object*, RA only in *whether* the scope is delineated, RS neither delineates nor reroutes, and RD and R0+ recompute globally: RD decodes the original chromosome once, whereas R0+ searches repeatedly within its budget.

The scoping rule of R1 is taken from the operation-level local rescheduling approach described in Section 2.1 and is reimplemented by us. The correspondence is as follows. The scoping rule "take the directly disturbed operations as the affected set, then expand forward along the within-job operation precedence" corresponds to AOR [18]. The two further components, *truncating the expansion at θ hops* and *mapping the operation set to the corridor reservations to be released*, are specified by us—the original setting has no reservation table, so this mapping has no counterpart there. The rerouting and replay components share the same implementation as R2, so that the two differ only in the scope object. R1 is therefore a baseline that transplants this scoping rule into our reservation setting; its results reflect how the rule performs in this setting, not how the original method performs in its own setting.

Budget convention. The methods are not compared at the same budget level. R1, R2, RS, RD, and RA are each a single deterministic repair that terminates when the run finishes; their runtime is determined by instance size and impact-set size rather than by a budget setting. Only R0+ terminates on a wall-clock budget. All results involving R0+ are therefore obtained under a given budget; the other five methods each run once and terminate, so they can be compared with one another directly. Wall-clock times are reported per batch and are not compared across batches.

Division of labor between the two batches. The two batches are not two samples of the same body of evidence; they answer different questions. The main batch answers *whether the phenomena exist and whether the mechanism can be identified in isolation*: under Class B disturbances, scoping on the operation-precedence graph yields an empty scope while the original schedule is infeasible (Section 6.2); the reservation impact set contains that scope and the fields outside the set remain invariant (Section 6.3); and only the scope object is replaced under the same rerouting procedure (Section 6.4). All three require full control over the instance parameters, especially the transport-to-processing time ratio that determines whether corridor contention arises, and the public instance families cannot provide such control. The second batch answers a different question: *whether these phenomena hold only on the layouts we constructed.* It is used only to test whether the main-batch results can be reproduced on the layouts of Lyu et al. (see Section 6.12, "Reproduction on the external-layout batch") and yields no new performance-comparison conclusions.

The two batches are analyzed jointly in only two places. The first is E4. The three same-size layouts of the main batch belong to one family with very small geometric differences, so two explanations—"the indicator is too coarse" and "the instances differ too little in geometry"—cannot be distinguished. Conversely, without a baseline on our own layouts matched parameter by parameter, the external layouts alone could not tell whether the variation in their impact-set share is large enough. The second batch serves to separate these two explanations. The second is E5: the E5 setup and parameters are identical in both batches, and pooling serves only to increase the sample size. The conclusions of all other sections are drawn from the main batch alone; the second batch only strengthens their external validity.

Statistical convention. The paper contains two inferential tests, both in Section 6.9: the difference between R2 and RA in rewrite fraction (the primary test) and the difference between R2 and RD in makespan. Both use the two-sided Wilcoxon signed-rank test, with the instance–seed pair as the pairing unit, i.e., on the difference between the two

Table 3. E1: scopes given by the two scoping rules under corridor blockage (10 random seeds per instance, 50 pairs in total). "All three hold" is the number of pairs in which the three conditions of Proposition 1 hold simultaneously ($T_{\text{impact}} = 0$, a nonempty seed set, and an infeasible original schedule after the blockage). $|R|$: occupancies in the whole table; |Src|: seed occupancies; |Cl|: occupancies in the reservation impact set; Cl/$|R|$: share of the reservation impact set in the whole table; means are over the 10 random seeds. funnel, high, and mid are the three spine layouts: funnel has one exit lane and one middle lane at the load/unload station, high has two exit lanes, and mid has two exit lanes and two middle lanes.

Instance	Seeds	All three hold	Mean $\|R\|$	Mean |Src|	Mean |Cl|	Cl/$\|R\|$
Small	10	10/10	28.6	5.6	16.9	0.59
Congested	10	10/10	85.0	15.0	48.4	0.57
funnel	10	10/10	149.1	15.5	85.4	0.57
high	10	10/10	150.5	15.5	88.3	0.59
mid	10	10/10	147.6	15.0	82.8	0.56
Total	50	50/50	—	—	—	—

methods on the same pair. After Holm correction for these two tests, the significance conclusion of each is unchanged. All other per-pair win/loss counts, means, and medians are *descriptive* statistics, and no significance claims are made for them. The primary test is accompanied by a Hodges–Lehmann effect-size estimate with a 95% bootstrap confidence interval (percentile method, 10^4 resamples, fixed random seed); dispersion is reported as the interquartile range. The tables of all other batches give only means, medians, and counts, without intervals.

6.2 E1: the operation-precedence graph yields an empty scope

This section tests whether Proposition 1 holds on the extended instances. Following the setup of Section 6.1, one busy corridor is blocked in each of the 50 pairs, the scope is delineated both on the operation-precedence graph and by the reservation impact set, and we check whether the original schedule is still feasible after the blockage. The results are shown in Table 3 and Figure 5 (left).

Table 3 shows that the three conditions of the proposition hold together in all 50/50 pairs: the affected operation set obtained on the operation-precedence graph is empty ($T_{\text{impact}} = 0$), the seed set and the reservation impact set are nonempty, and the original schedule is infeasible after the blockage. The first condition follows directly from the convention of Definition 2 for Class B; the substantive one is the third, namely that the original schedule *is* indeed infeasible. Hence, in these pairs, the inference "the operation-precedence graph yields an empty scope, so no rescheduling is needed" never holds.

The table also shows that |Cl| grows with the instance (16.9, 48.4, 82.8–88.3), a direct consequence of the larger total number of occupancies. Cl/$|R|$ lies within 0.56–0.59 on all five instances, and the variation between instances is of the same order as that between random seeds of the same instance: the per-instance interquartile range is 0.024–0.049, and over all 50 pairs the median is 0.571 and the range 0.500–0.655.

The results of this section support contribution C1: under Class B disturbances, scoping on the operation-precedence graph yields an empty scope although the original schedule is already infeasible.

6.3 E2: containment

This section tests whether two properties of the reservation impact set hold in the experiments. **E2a, structural closure** (Proposition 2): no dependency edge leads from the reservation impact set to an unfinished occupancy outside it. This

Table 4. E2: containment (10 random seeds per instance). E2a: no dependency edge leads from the reservation impact set to an unfinished occupancy outside it; "Single-pass feasible": feasibility is restored by rerouting only inside the impact set, without scope escalation; E2b: the corridor, time interval, and vehicle of *every* occupancy outside the set are unchanged. For funnel, high, and mid, see Table 3.

Instance	E2a closed	Single-pass feasible	E2b outside set invariant per occupancy
Small	10/10	10/10	10/10
Congested	10/10	10/10	10/10
funnel	10/10	10/10	10/10
high	10/10	10/10	10/10
mid	10/10	10/10	10/10
Total	50/50	50/50	50/50

is a property of the graph, guaranteed directly by the definition; it checks that the implementation agrees with the definition. **E2b, invariance outside the set** (Proposition 3): after single-pass rerouting inside the impact set, every occupancy outside the set keeps its corridor, interval, and vehicle. This is a property after repair; it checks whether the premises of that proposition are met in the experiments. One premise requires that, before the repair starts, every occupancy outside the set can be written back into the new reservation table with its original fields (called *pre-commit* below). If an occupancy outside the set overlaps the blockage window, the pre-commit fails, which means that the seed set has missed an occupancy overlapping the blockage.

The results are shown in Table 4. In all 50 pairs, the impact set is closed; single-pass rerouting inside the impact set (without scope escalation) restores feasibility; and every occupancy outside the set keeps its original values, i.e., no pre-commit failure occurs. Because transport legs spanning t_{now} lie inside the impact set, E2b does not compare their already traversed segments; we therefore separately check every occupancy with $t^e \leq t_{\text{now}}$ one by one, and none is rewritten in any of the 50 pairs.

Together with E1, the results of this section support the part of contributions C1 and C2 that concerns the scope object: the reservation impact set is uniquely determined by the dependency edge set and closed with respect to it, rerouting only inside it restores feasibility, and the occupancies outside it keep their original values.

6.4 E3: boundary ablation

This section tests whether, with the same rerouting procedure, replacing only the scope object changes the feasibility rate. R1 and R2 use the same initial solution, the same decision time, the same random seeds, and the same blockage window, and both perform only single-pass rerouting; **the only difference is which occupancies are rewritable**. Scope escalation is disabled in this section: if it were enabled, R1 could restore feasibility after escalating to all unfinished occupancies, and the measurement would no longer reflect the difference between scope objects. The results are shown in Table 5 and Figure 5 (right).

Table 5 shows that in all 50 pairs, R1's rewritable set is empty whereas R2's is nonempty; R1 is infeasible in every pair, and R2 restores feasibility in every pair. When R1's rewritable set is empty, single-pass rerouting changes no occupancy, so the original schedule is kept as it is and still occupies the blocked corridor. R1 is therefore infeasible because it decides that no rescheduling is needed, not because its repair is poor. The runtime of R2's single-pass rerouting has a median of 3.2 ms and a maximum of 12.8 ms.

Table 5. E3: boundary ablation (no scope escalation; 10 random seeds per instance). "R1 empty, R2 nonempty" is the number of pairs in which R1's rewritable set is empty whereas R2's is nonempty; the last two columns are means over the 10 random seeds. For funnel, high, and mid, see Table 3.

Instance	R1 empty, R2 nonempty	R1 feasible	R2 feasible	Mean $\lvert U_{R2}\rvert$	R2 runtime /ms
Small	10/10	0/10	10/10	16.9	0.66
Congested	10/10	0/10	10/10	48.4	2.84
funnel	10/10	0/10	10/10	85.4	5.09
high	10/10	0/10	10/10	88.3	3.40
mid	10/10	0/10	10/10	82.8	3.96
Total	50/50	0/50	50/50	—	—

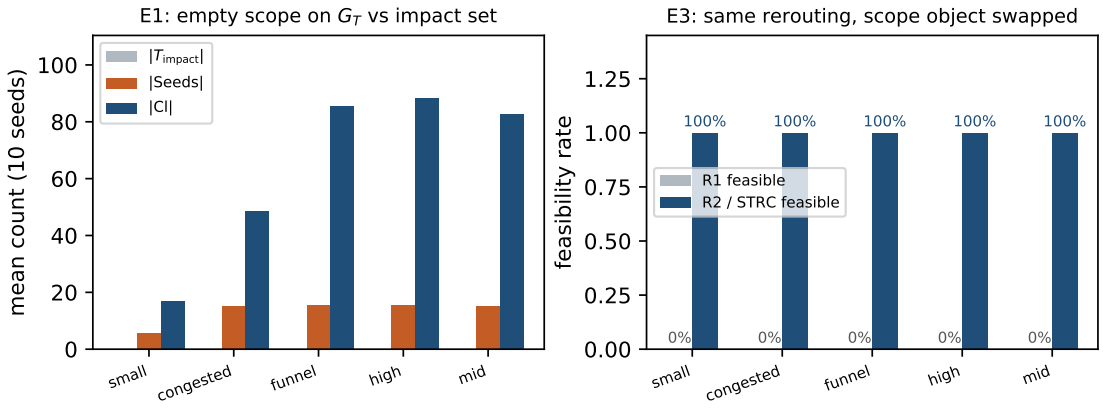


Fig. 5. Results of E1 and E3 (10 random seeds per instance). Left: for each instance, the number of affected operations on the operation-precedence graph, the number of seed occupancies, and the number of occupancies in the reservation impact set; bar heights are means over the 10 random seeds, and the per-instance interquartile range of |Cl| is 2–8. Right: feasibility rates of R1 and R2 under single-pass rerouting without scope escalation, counted per instance–seed pair.

Our corridor edges are equally weighted, i.e., every corridor has the same traversal time. To test whether the above results depend on this setting, we rerun E1 and E3 on the same 50 pairs under three non-uniform edge weightings: first, each corridor's traversal time is multiplied by an independent lognormal factor (geometric mean 1); second, corridors farther from the load/unload station are slower, up to 1.8 times the original at the farthest; third, each corridor randomly takes half or twice its original traversal time. Under all three weightings, the three conditions of E1 still hold in every pair and R1 is still infeasible everywhere; R2 restores feasibility in all pairs under the first two weightings and fails in 1 pair under the third (49/50). Hence, neither the empty scope given by the operation-precedence graph nor R1's failure to restore feasibility changes with the edge weights.

The results of this section support contributions C2 and C3: replacing only the scope object changes the feasibility rate from 0/50 to 50/50.

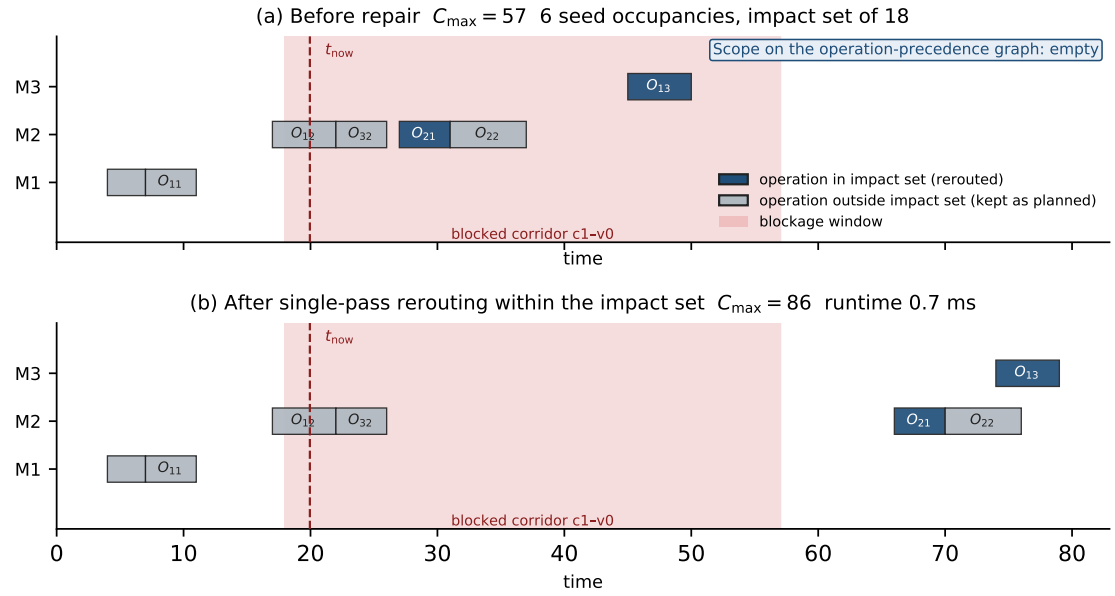


Fig. 6. Case Gantt chart ($3 \times 3 \times 2$ small instance, run seed 42, busy-corridor blockage). (a) Original schedule before the blockage, colored by membership in Cl; (b) after single-pass rerouting. The affected set on the operation-precedence graph is empty; blue operations are those whose transports in the impact set are rerouted. The figure only illustrates the form of the local change.

6.5 Case Gantt chart

E1–E3 report counts and feasibility rates over 50 pairs. To show on the operation timeline a case in which "the operation-precedence graph yields an empty scope, but scoping on the reservation impact set can repair it," Figure 6 gives a concrete example (the small instance is chosen because its Gantt chart shows every operation clearly): the instance is the $3 \times 3 \times 2$ small instance (3 jobs, 3 machines, 2 AGVs), the run seed is 42, the disturbance is a busy-corridor blockage, and all other settings are as in E3.

In this case the set of affected operations on the operation-precedence graph is empty ($T_{\text{impact}} = \varnothing$), whereas there are 6 seed occupancies and 18 occupancies in the reservation impact set. Figure 6(a) is the machine Gantt chart of the original schedule before the disturbance: blue operations have transport occupancies in the impact set (to be rerouted), and gray operations lie outside the impact set (pre-committed and kept as planned); the red dashed line marks t_{now}, and the light red band marks the blockage window. Figure 6(b) shows the schedule after repair: the operations in the impact set are rearranged on the timeline, and the schedule regains feasibility under the blockage, with $C_{\max} : 57 \longrightarrow 86$ and a sub-millisecond runtime. This figure only illustrates the form of the local change; it is not used to compare makespans.

6.6 E4: can network-structure indicators predict the impact-set size? (exploratory)

This section tests whether indicators computed from the network alone, independent of the current schedule, can predict the size of the reservation impact set. We consider three indicators: first, the *LU cut*, i.e., the minimum number of corridors that must be cut to disconnect the load/unload station from all machines (the load/unload minimum cut); second, the *funnel share*, i.e., among the shortest paths from the load/unload station to the machines, the travel time on

corridors that every path must use, as a share of the mean path travel time; third, corridors per node, i.e., the ratio of corridors to nodes. We call these three indicators *static network indicators* below. The disturbance in this section blocks one corridor in the LU cut; all other settings are as in Section 6.1. The two batches together contain 8 layouts, each with 10 random seeds. The results are shown in Table 6.

The last three rows of Table 6 show that the impact-set shares of the three spine layouts we constructed lie between 0.446 and 0.497, a range of only 0.051; their geometric differences are also small (9–11 nodes, 8–12 corridors, and an LU cut of only 1 or 2). Both "the indicators are too coarse to reflect layout differences" and "these layouts differ too little" are consistent with this, **so our three layouts alone cannot tell whether the static network indicators are informative**, and the second batch is needed.

The five layouts of the second batch lie on only two grids (Section 6.1), so they can be used only to examine the machine-configuration dimension. On the same 4×4 grid, changing only the machines from 4 to 5 (layout 2 to layout 3) lowers the median impact-set share from 0.512 to 0.305, a difference of 0.207; on the same 5×5 grid, three machine configurations (layouts 4–6) differ by 0.075. Our three layouts, whose networks all differ, differ from one another by only 0.051; the former two differences are about four and one and a half times as large. The comparison is not symmetric: the layouts of Lyu et al. change both the number and the positions of machines, whereas our three layouts change only the network and differ little in geometry, so 0.051 is only a lower bound on the variation caused by changing the network, and the comparison does not show that machine configuration matters more than network geometry. **Hence, with the network fixed, changing only the machine configuration already changes the impact-set share markedly; the machine-configuration dimension cannot be ignored.**

In the table, the LU cut is 2 and the funnel share is 0 on all five external instances. The reason is that both grids place the load/unload station at a grid corner: with only up, down, left, and right moves, a corner has only two corridors, so the LU cut is fixed at 2; and the shortest paths from the station to the machines share no corridor, so the funnel share is 0. The Liu 2023 layouts in the van Os toolkit (not included in this batch because they allow diagonal moves) place the station at the midpoint of a grid edge, with out-degree 3. The LU cut on practical layouts thus takes only a few values determined by the station position and does not vary continuously with network geometry. The impact-set share varies by 0.207 on these instances while these two indicators do not change, so they cannot explain this variation. The third indicator (corridors per node) takes only two values over the whole batch (23/16 and 38/25) and likewise cannot explain the variation within one grid. These indicators were designed for the adjustable load/unload exit bottleneck of spine layouts and degenerate to constants on practical layouts. **Hence the static network indicators are constant on the layouts of Lyu et al. and cannot explain the variation in impact-set size.**

Conclusion: on the 5 instances of the second batch and the three layouts we constructed, the static network indicators do not explain the impact-set size; the impact-set size more likely depends on the spatial distribution of transport demand over the network than on how many routes the network can offer, and specific candidate quantities are given in Section 7.2, "Future work".

6.7 E5: trade-off against global rescheduling

This subsection tests whether a *crossover* exists, that is, whether bounded repair dominates under tight budgets while warm-started global rescheduling overtakes it in $C_{\max}$ once the budget is relaxed. The baseline is R0+, warm-started global rescheduling that does not rewrite finished occupancies, with budgets $\{0.2, 1, 2\}$ s; R2 runs only once per pair, and its result does not depend on the budget. The instances come in two batches: this subsection reports the small and the congested instance constructed in this paper, with random seeds $\{42, 7, 2024\}$; Section 6.12 repeats the experiment

Table 6. E4: static network indicators and the reservation impact-set share (one corridor in the LU cut blocked; 10 random seeds per layout, median Cl/|R|). All instances share the number of jobs, the number of AGVs, and the other generation parameters, and differ only in network and machine configuration. "Nodes/corridors" gives the numbers of nodes and corridors. "Range" is the maximum minus the minimum impact-set share among the layouts of one group; the 4 × 4 group has only 2 layouts and our group only 3, so the ranges allow only a rough comparison.

Source and grid	Layout	Machines	Nodes/ corridors	LU cut	Funnel share	Cl/|R|	Range
Lyu et al., 4 × 4 grid	Layout 2	4	16/23	2	0	0.512	
Lyu et al., 4 × 4 grid	Layout 3	5	16/23	2	0	0.305	0.207
Lyu et al., 5 × 5 grid	Layout 4	6	25/38	2	0	0.385	
Lyu et al., 5 × 5 grid	Layout 5	7	25/38	2	0	0.422	
Lyu et al., 5 × 5 grid	Layout 6	8	25/38	2	0	0.460	0.075
Ours	funnel	4	9/8	1	0.286	0.491	
Ours	high	4	10/10	2	0.286	0.497	
Ours	mid	4	11/12	2	0.286	0.446	0.051

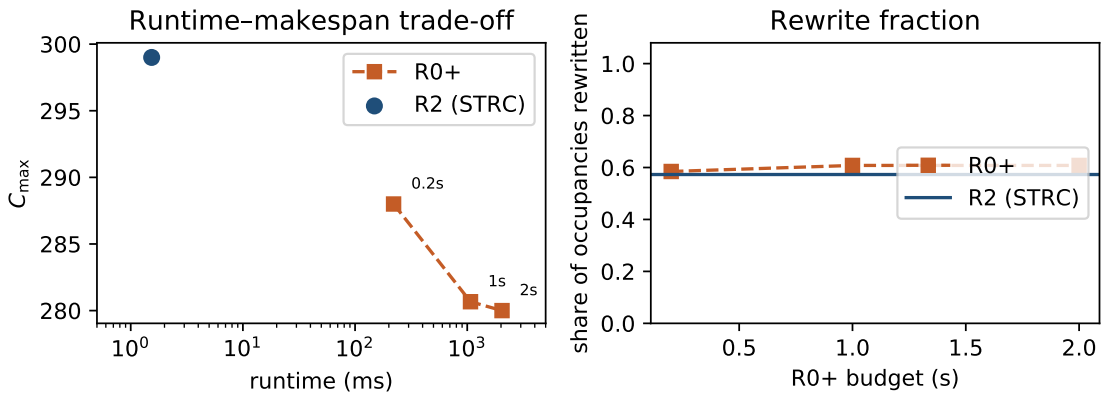


Fig. 7. E5: budget sweep of R0+ (which does not rewrite finished occupancies) versus bounded repair (congested instance, mean over 3 random seeds). Left: runtime versus C_{max}. Right: occupancy rewrite fraction.

with the same parameters on two external layouts. The two batches together comprise 4 instances, 12 instance–seed pairs, and 36 budget points.

The results on the two instances constructed in this paper are shown in Table 7 and Figure 7. Four points can be observed.

First, R0+ has a lower C_{max} than R2 at every budget point, and the two curves in Figure 7 (left) do not intersect; together with the external-layout batch, the R0+ win/tie/loss over the 36 budget points is 35/1/0.

Second, there is no crossover within the tested range, and this extends to larger budgets. The C_{max} of R2 does not change with the budget; R0+ reports the best-so-far value per generation, and with the random seed fixed, a larger budget only lets the same search trajectory run for more generations, so its C_{max} is monotonically non-increasing. Hence, once the order of the two is settled at the smallest budget level, increasing the budget cannot reverse it. At the smallest level, among the 12 pairs, R0+ is strictly better in 11, ties in 1, and is worse in none; both the monotonicity

Table 7. E5 comparison with warm-started global rescheduling that does not rewrite finished occupancies (mean over 3 random seeds). This table covers only the two instances constructed in this paper: 2 instances × 3 random seeds × 3 budget points, i.e., 6 independent pairs; for the external-layout batch, see Section 6.12. "Rewrite fraction" is the share of occupancies in the whole table that are rewritten. R2 runs only once per pair and is not affected by the budget, so its two columns have the same values in the three budget rows of an instance, all from the same run.

Instance	Budget/s	R0+ $C_{\max}$	R2 $C_{\max}$	Rewrite fraction R0+ / R2
Congested	0.2	288.0	299.0	0.58 / 0.57
	1.0	280.7	299.0	0.61 / 0.57
	2.0	280.0	299.0	0.61 / 0.57
Small	0.2	78.3	88.7	0.60 / 0.60
	1.0	78.3	88.7	0.60 / 0.60
	2.0	78.3	88.7	0.60 / 0.60

and the budget invariance of R2 were checked pair by pair on both batches. Two statements should be kept apart: "increasing the budget cannot reverse the order" is guaranteed by monotonicity and holds for any instance; "R0+ is no worse than R2 at the smallest budget" is only an observation on these 12 pairs and need not hold on other instances. The 36 budget points come from 4 instances and 12 pairs, and the three budget points of a pair belong to the same search trajectory, so they are not independent; this subsection therefore performs no significance test.

Third, this extension does not apply to lower budgets. R0+ needs at least one decoding per candidate evaluated, and one decoding takes about 4.16 ms (median); with a population of 40 individuals, evaluating the initial population alone takes about 0.17 s, close to the smallest tested budget of 0.2 s. Below 0.2 s, R0+ may therefore be unable to evaluate even one generation, and the order may reverse, whereas R2 needs only milliseconds and is feasible in all pairs; this range was not tested.

Fourth, bounded repair yields a worse makespan at every budget point, but its runtime is two to three orders of magnitude lower (3.2 ms for single-pass rerouting versus budgets of 0.2–2 s; see Figure 7, left). This runtime gap comes from a single rerouting pass that does not restart the search; it does *not* come from scoping on the impact set, as Section 6.9 (comparisons within the millisecond range) confirms with the RA baseline of equal runtime. The occupancy rewrite fractions of both methods are about 0.6 (last column of Table 7 and Figure 7, right).

In the makespan comparison above, neither method rewrites finished occupancies. R0+ uses the fixed-prefix decoding of Section 5.3; if at t_{now} a vehicle has already set off empty to pick up a job, while the preceding operation of that job is not yet finished and the pickup time must be redetermined, R0+ keeps the original vehicle, leaves the stretch already traveled unchanged, and replans only the part after t_{now}. A point-by-point check shows that R0+ rewrites finished occupancies 0 times at each of the 9 budget points of both instances, and bounded repair likewise rewrites none in the same pairs. The makespan gap between the two therefore contains no contribution from "rewriting finished occupancies"; it comes from the advantage of global reassignment and resequencing over a single rerouting pass.

Conclusion: when a millisecond-scale response is required, bounded repair restores feasibility; when a response time of seconds is acceptable, warm-started global rescheduling that does not rewrite finished occupancies achieves a better makespan. The value of the reservation impact set does not lie in the makespan; its intended use is recovery that must respond within milliseconds.

Table 8. E6 disturbance type × scope object (50 pairs per type, 200 measurements in total). Slowdown and blockage have the same seed occupancies and therefore yield the same reservation impact set, so there are 150 *independent* scoping measurements. $|R^\circ|$ is the number of occupancies unfinished after t_{now}. "R1 empty" is the number of pairs in which scoping on the operation-precedence graph deems the disturbance to require no rescheduling. The last column uses no scope escalation: single-pass rerouting for Class B, and vehicle swapping or machine reassignment for Class A.

Disturbance	Class	R1 empty	Mean $\lvert U_{R1}\rvert$	Mean $\lvert\text{Cl}\rvert$	$\text{Cl}/\lvert R^\circ\rvert$	R2 feasible
Corridor blockage	B	50/50	0.0	64.4	0.92	50/50
Corridor slowdown	B	50/50	0.0	64.4	0.92	50/50
Vehicle failure	A	0/50	31.1	60.6	0.85	50/50
Robotic-arm failure	A	0/50	42.8	65.9	0.94	50/50

6.8 E6: crossing disturbance types with scope objects

E1–E3 consider only one disturbance, corridor blockage. This subsection tests whether the scoping and repair conclusions of E1–E3 still hold under other disturbance types: on the same 50 pairs we add three further types, namely corridor slowdown, vehicle failure, and robotic-arm failure.

What is measured. For scoping, we report for all four disturbance types the size of the affected set on the operation-precedence graph, the size of the reservation impact set, and whether the latter contains the former. For repair, we run the repair for all four types. Class B still uses rerouting only: the blockage window is written into the reservation table, while a slowdown doubles the traversal time of the corridor during the slowdown window; outside the window traversal is unchanged, and the corridor remains passable throughout. Class A additionally allows vehicle swapping or machine reassignment, without changing the processing order; whether the rewritable scope is escalated does not affect its action set.

The results are shown in Table 8. Four points can be observed.

First, scoping on the operation-precedence graph is empty for both Class B disturbances in every case and nonempty for both Class A disturbances in every case; that is, **the A/B dichotomy holds without exception over the** 150 ***independent* scoping measurements**.

Second, over all 100 Class A measurements, $U_{R1} \subseteq \text{Cl}$; the containment is strict in 87/100 pairs, and the two sets are equal in the remaining 13 pairs. For the robotic-arm failure row, containment holds *by construction*: the seed rule of Definition 3(c) takes the entire suffix along the job chain, whereas U_{R1} is truncated at $\theta = 2$ (Section 6.1); this row thus only verifies that the implementation agrees with the definition. For the vehicle failure row, containment is an empirical result. More occupancies become rewritable on average: under vehicle failure, from 31.1 with scoping on the operation-precedence graph to 60.6 with scoping on the reservation impact set; under robotic-arm failure, from 42.8 to 65.9. **Hence, on Class A, switching to the reservation impact set never *misses* an occupancy already included by the operation-precedence graph, at the price of a larger rewritable scope.**

Third, slowdown and blockage give the same impact-set size pair by pair on all 50 pairs. The reason is that the two share the same seed rule (unfinished occupancies overlapping the disturbed interval), which by construction yields the same reservation impact set; scoping on the operation-precedence graph also yields an empty scope for slowdown. **Hence slowdown and blockage behave identically in scoping.**

Fourth, repair is feasible for all four disturbance types in every case (last column of Table 8): vehicle failure is handled by vehicle swapping, robotic-arm failure by machine reassignment, and both Class B disturbances are fully repaired by single-pass rerouting without triggering scope escalation. A slowdown only doubles the traversal time during the

slowdown window, and the corridor remains passable, so it still differs physically from "full no-entry followed by a detour". **Hence bounded repair restores feasibility under all four disturbance types.**

Impact-set size. Under corridor blockage, the reservation impact set contains about 92% of the occupancies unfinished at t_{now}; in other words, compared with "including all unfinished occupancies in the rewrite", it leaves out fewer than 10% of them. This share has a different denominator from Cl/$|R|$ in E1 (0.56–0.59): the latter is taken over all occupancies in the table, about two fifths of which are finished before t_{now}.

Conclusion: the results of this subsection extend the conclusions of E1–E3 on C1 and C2 from corridor blockage to four disturbance types.

6.9 E7: comparisons within the millisecond range

The baseline in E5 is a warm-started population-based search with a budget of 0.2–2 s. Using it as the only baseline leaves a question open: *how much of the "two to three orders of magnitude lower" result is due to the reservation impact set, and how much merely to the longer runtime of the baseline?* This subsection adds baselines whose runtime lies between those of bounded repair and warm-started population-based search, together with a baseline that changes only the rewritable scope.

- **RS (global right shift).** Without changing routes, assignments, or processing orders, RS postpones all unfinished occupancies as a whole by a delay Δ, where Δ is just large enough to move every movable occupancy on the blocked corridor past the blockage interval. Like R2, it does not rewrite finished occupancies (those with $t^e \le t_{\text{now}}$ are immovable). Because the shift is uniform, every time gap stays the same or grows, and all constraints are of the "$\ge$" type, so a feasible solution is obtained without rerouting. RS represents one extreme of "*not scoping on the impact set*" and performs no rerouting at all (Section 2.1).
- **RD (re-decoding of the original chromosome).** RD keeps the machine assignments and processing orders and runs the same fixed-prefix decoding as R0+ on the routing layer into which the blockage has been written. It is the *fastest global recomputation* in our implementation; because it does not change the chromosome, it is faster than R0+ but has less freedom to reassign and resequence.
- **RA (no scope restriction by the impact set, but with rerouting retained).** RA includes all occupancies *unfinished* after t_{now} in the rewrite and then runs a rerouting replay *identical* to that of R2. It represents the other extreme of "not scoping on the impact set": unlike RS, it retains rerouting, so its only difference from R2 is the set of rewritable occupancies. The two use the same rerouting procedure, neither rewrites finished occupancies, and both face the same blockage. RA is therefore the *only* baseline in this paper that isolates the effect of the reservation impact set; the difference between the two methods involves no difference in the rerouting procedure or the implementation. Since RA's rewritable set is already maximal, it needs no scope escalation after failure.

E3 compares two *different* scope objects (the operation-precedence graph and the reservation impact set), whereas RA addresses the question *whether the rewritable scope needs to be restricted by the impact set at all.*

The results are shown in Table 9. Four points can be observed.

First, RS needs only 1.42 ms, about half the time of R2, and is the only method among those returning a feasible plan that is faster than bounded repair (R1, which scopes on the operation-precedence graph, is faster, but its feasibility rate is only 0/50, so it is not a candidate method). The price is a mean makespan degradation of 58.8% (versus 49.8% for R2), a pair-by-pair result of 27/16/7 (R2 win/tie/loss), and more rewritten occupancies (0.627 vs. 0.540), because the uniform shift changes the times of every unfinished occupancy. Making the right shift *selective*, that is, delaying

Table 9. E7 comparisons within the millisecond range (50 pairs; same setup as E3: busy-corridor blockage, R2 without scope escalation). All four methods are feasible in 50/50 cases. "Finished-occupancy violations" is the number of pairs in which finished occupancies were rewritten; "Rewritable" is the share of occupancies unfinished after t_{now} that the method includes in the rewrite; the last column gives the method's $C_{\max}$ win/tie/loss against R2. "Mean degradation" is the mean over the 50 pairs of $(C_{\max}(\mathcal{S}') - C_{\max}(\mathcal{S}^0))/C_{\max}(\mathcal{S}^0)$, where $\mathcal{S}^0$ is the pre-disturbance schedule and $\mathcal{S}'$ the repaired one. RA and R2 differ only in which occupancies are rewritable, so the entire difference between these two rows is attributable to the reservation impact set. Table 7 contains only two instances, the small and the congested one, which are not the same instance set as the 50 pairs here, so the values of the two tables should not be compared directly.

Method	Finished-occupancy violations	ms (median)	Mean degradation	Rewrite fraction	Rewritable	W/T/L vs. R2
RS global right shift	0/50	1.42	58.8%	0.627	—	7/16/27
RD re-decoding	0/50	4.16	49.1%	0.618	—	22/14/14
RA no impact-set scoping	0/50	2.76	49.9%	0.604	1.00	0/46/4
R2 bounded repair	0/50	2.74	49.8%	0.540	0.92	—

only the vehicles actually affected and leaving the rest unchanged, would first require determining which occupancies are affected, which is precisely the scope this paper delineates. **Hence the right shift is fast because it gives up selectivity, and it pays with both a worse makespan and more rewriting.**

Second, the median runtime of RD is 4.16 ms, 1.5 times that of R2. This result does not depend on the search configuration: any decoding-based search needs at least one decoding per candidate evaluated. **Hence, even with the baseline replaced by the fastest global recomputation in the same implementation, bounded repair is still faster**; the "two to three orders of magnitude lower" result in E5 is not an artifact of an overly slow baseline.

Third, over the whole table, R2 versus RD is 14/14/22 (win/tie/loss), with mean degradations of 49.8% and 49.1%; the mean paired difference is +0.006 with a bootstrap 95% interval of $[-0.007,\ 0.019]$, which contains 0, and the two-sided Wilcoxon signed-rank test gives $p = 0.12$, so the difference is not significant. RD rewrites finished occupancies in 0/50 pairs: if at t_{now} a vehicle has already set off empty to pick up a job while the preceding operation of that job is not yet finished, RD keeps the original vehicle, leaves the stretch already traveled unchanged, and replans only the part after it. RD's slight edge in the pair-by-pair count therefore stems from the advantage of global rerouting under the same chromosome over a single rerouting pass, not from rewriting finished occupancies. RD's rewrite fraction is 0.618, higher than R2's 0.540. **Hence, without rewriting finished occupancies, RD and R2 are indistinguishable in makespan, but RD rewrites more.**

Fourth, RA and R2 differ only in the set of rewritable occupancies, so this pair directly gives the benefit of scoping on the impact set. On average, the reservation impact set includes only $1 - 0.92$ fewer unfinished occupancies than the set of all unfinished occupancies (0.80–1.00 per pair; in the 6 pairs with value 1.00, the impact set *equals* the set of all unfinished occupancies, and the two methods give identical results). This difference of less than one tenth lowers the rewrite fraction from 0.604 to 0.540: a mean reduction of 0.064, a median reduction of 0.029, a pair-by-pair result of 31/18/1 (R2 more stable/tie/less stable), and a two-sided Wilcoxon signed-rank $p = 8.8 \times 10^{-7}$. The Hodges–Lehmann estimate of the effect size is 0.037, with a 95% bootstrap confidence interval of $[0.018,\ 0.074]$; the mean paired difference 0.064 has interval $[0.039,\ 0.093]$. The interquartile range of the rewrite fraction is 0.073 for R2 and 0.062 for RA. Because the two methods tie exactly in 18 of the 50 pairs, the lower end of the confidence interval for the *median* paired difference is 0.

Excluding the 6 pairs with identical results leaves 44 pairs, with a pair-by-pair result of 31/12/1 and a mean reduction of 0.073 (under both counts, the Wilcoxon test has 32 pairs with nonzero differences); the reduction reported over all 50

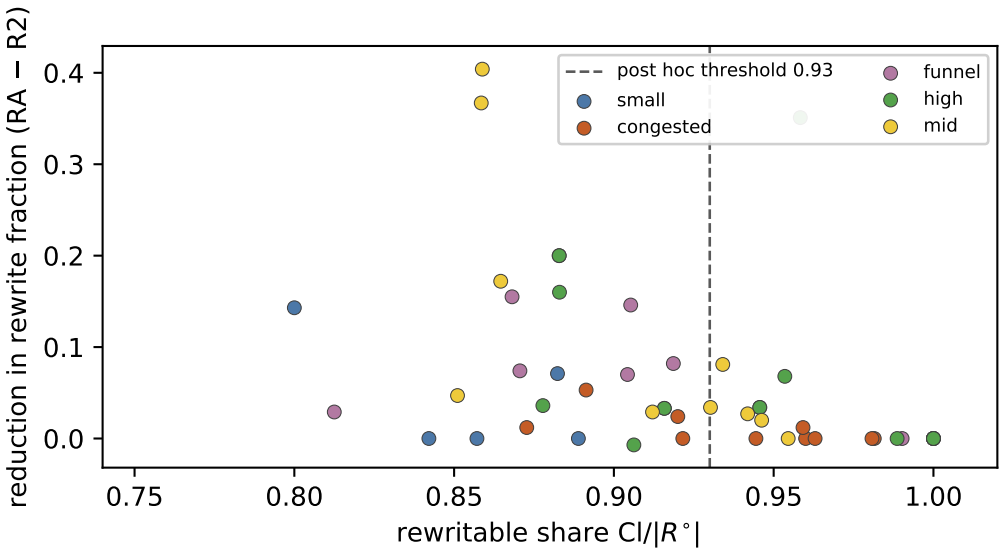


Fig. 8. E7 post hoc rule: rewritable share versus reduction in rewrite fraction (50 pairs, R2 vs. RA). The dashed line marks the in-sample threshold 0.93; positive values on the vertical axis mean a lower rewrite fraction with impact-set scoping.

pairs is thus conservative. The direction of the reduction is highly consistent, but its *magnitude is small*; in the only pair where R2 is worse, R2 rewrote 0.007 more. Meanwhile, the two methods have nearly identical runtimes (2.74 vs. 2.76 ms) and nearly identical makespans (mean ratio 0.999; pair-by-pair R2 win/tie/loss of 4/46/0, never worse, with 0.970 in the best pair). **Hence scoping on the impact set reduces rewriting, and this can be established only by comparing R2 with RA.**

The size of the reduction depends on how many occupancies the impact set excludes. The more it excludes, the larger the reduction; however, the excluded occupancies themselves account for only part of the gap 0.604 − 0.540; the rest comes from changes in the rerouting outcome once the rewritable set shrinks. On the instance where the impact set nearly equals the set of all unfinished occupancies (the congested instance, rewritable share 0.94), the reduction is only 0.010; on the instance that excludes more occupancies (the mid layout, 0.91), the reduction reaches 0.118. Two instances can only indicate a direction. To see the per-pair distribution, Figure 8 plots the 50 pairs with the rewritable share $\mathrm{Cl}/|R^{\circ}|$ on the horizontal axis and the reduction in rewrite fraction of R2 relative to RA on the vertical axis, with an in-sample threshold at $\mathrm{Cl}/|R^{\circ}| = 0.93$: the 23 pairs above it have a mean reduction of 0.029, and the 27 pairs below it have a mean reduction of 0.094. All pairs with a share above 0.97 show zero reduction, and two cases occur among them. In the 6 pairs with share 1.00, the impact set is exactly the set of all unfinished occupancies, so R2 and RA perform identical computations and the reduction is necessarily zero; in the remaining pairs, the impact set excludes a few occupancies and the reduction is still zero, which is an empirical result. The threshold is chosen on this sample to answer "when does scoping on the impact set still pay off"; it is not meant to be extrapolated, and it differs from a structural prediction of the impact-set size (Section 6.6).

Conclusion: under the two constraints that "finished occupancies are not rewritten" and that "the response must be within milliseconds", the four methods together show the following. Equal runtimes of R2 and RA show that the

Table 10. Sweep over disturbance magnitude (congested instance, averaged over 10 random seeds; bounded repair with scope escalation; R0+ budget 2 s). The per-pair runtime of bounded repair ranges over 2.2–23.8 ms; the per-level interquartile range of the R0+ $C_{\max}$ lies between 8 and 76.

φ	Bounded repair feasible	Bounded repair ms	R0+ $C_{\max}$
0.10	100%	9.1	103.5
0.25	100%	11.1	107.0
0.50	100%	8.6	111.8
0.75	100%	8.4	125.8
1.00	100%	10.7	148.1

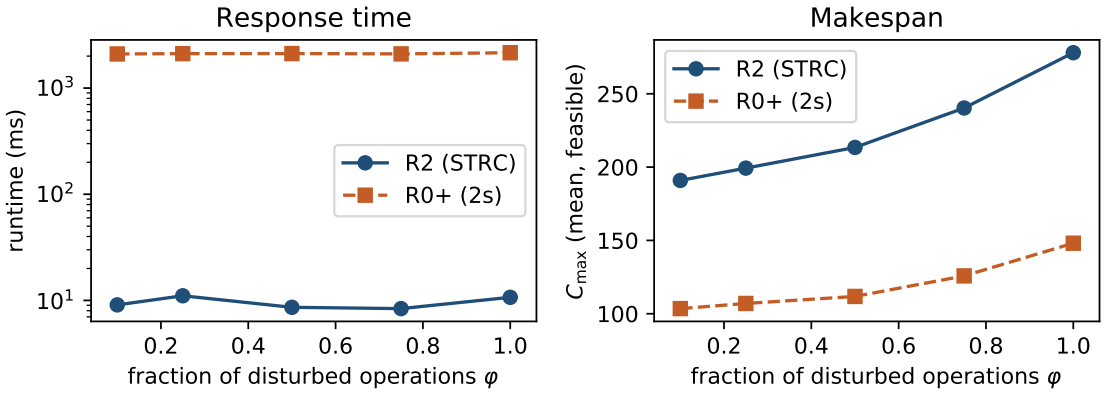


Fig. 9. Sweep over disturbance magnitude (congested instance, 10 random seeds): runtime (log scale) and makespan of bounded repair and R0+ as functions of φ. Curves show per-level means; dispersion is measured as in Table 10 (interquartile range).

millisecond-scale response comes from a single rerouting pass that does not restart the search, not from the reservation impact set. Their equal makespans show that the impact set is not an optimization device. R2 has the lower rewrite fraction because occupancies outside the impact set keep the original plan by Proposition 3 and are not rewritten. **The benefit of scoping on the impact set lies in the rewrite volume, not in speed or makespan**: at equal runtime and makespan, the share of occupancies in the table that are rewritten is 0.604 without impact-set scoping (RA) and 0.540 with it (R2), a difference of about 6 percentage points; that is, RA rewrites about 12% more occupancies than R2. If a runtime of seconds is acceptable, warm-started global rescheduling R0+ obtains a shorter makespan $C_{\max}$ (E5, Section 6.7), but its runtime then exceeds the millisecond range.

6.10 Sweep over disturbance magnitude

This subsection tests whether bounded repair still restores feasibility, and whether its runtime stays in the millisecond range, as the disturbance grows. The instances are the congested and the small instance, with 10 random seeds each; the share φ of disturbed operations takes 0.10, 0.25, 0.50, 0.75, and 1.00, where $\varphi = 1$ means that all operations after t_{now} are disturbed. Unlike in E3 and E5, bounded repair in this subsection has scope escalation enabled; R0+ has a budget of 2 s.

The results on the congested instance are shown in Table 10 and Figure 9. Three points can be observed. First, bounded repair restores feasibility in every pair at all five levels of φ, and likewise in every pair on the small instance.

Table 11. E8 sweep over instance size (7 levels × 10 random seeds). "Impact set" is the reservation impact set as a share of the *whole table*; the attainable upper bound of this share is $|R^\circ|/|R|$, whose mean in this batch is about 0.61. With the rewritable set as the denominator, the share is 0.78–0.91 (see the text). Runtimes are medians, and the "RD/R2" column is the *ratio of the two median columns*. When the text discusses monotonicity, it uses ratios computed per pair and then averaged per level; the two measures rise and fall alike across levels but differ in value. The last column gives the win/tie/loss of R2 against RS on $C_{\max}$.

Size	$\lvert R\rvert$	Impact set	R2 ms	RD ms	RD/R2	R2 vs. RS
$8 \times 4 \times 4$	148	0.561	2.9	4.4	1.5×	7/1/2
$10 \times 5 \times 5$	172	0.485	3.2	6.1	1.9×	9/0/1
$12 \times 6 \times 6$	230	0.481	4.5	10.7	2.4×	7/1/2
$14 \times 7 \times 7$	281	0.492	6.1	14.5	2.4×	8/0/2
$16 \times 8 \times 8$	291	0.534	6.4	16.4	2.6×	10/0/0
$18 \times 9 \times 9$	356	0.535	8.2	26.4	3.2×	10/0/0
$20 \times 10 \times 10$	402	0.476	10.4	34.2	3.3×	10/0/0

Second, the runtime of bounded repair does not rise with φ: the level means on the congested instance are 8.4–11.1 ms, the per-pair values 2.2–23.8 ms, and 10.7 ms at $\varphi = 1$; on the small instance it is about 1 ms. Third, R0+ attains the better $C_{\max}$ at every level, consistent with E5, but it uses its full 2 s budget, about two orders of magnitude more time.

Conclusion: within the tested range of disturbance magnitudes, bounded repair always restores feasibility, and its runtime stays on the order of ten milliseconds without growing with the disturbance magnitude.

6.11 E8: extrapolation to larger instances

The previous subsection swept the magnitude of the *disturbance*, with the instance fixed at 8 jobs and 4 machines. This subsection fixes the disturbance setup and enlarges the instance itself, testing whether the impact-set share approaches 1 as the instance grows, which would make "bounded" meaningless in practice; runtime is recorded as well.

The size sequence uses $2k$ jobs, k machines, and k vehicles for $k = 4, \ldots, 10$, that is, from $8 \times 4 \times 4$ to $20 \times 10 \times 10$: 7 levels × 10 random seeds = 70 pairs. The congestion level, heterogeneity, flexibility, transport-to-processing time ratio, and the load/unload-station minimum cut and remote cut are all calibrated to the same targets, so *only the size changes*. The three baseline methods are defined as in Section 6.9. Every level's instance is generated from the same random seed 42, at a single congestion level; E6 and the disturbance-magnitude sweep were not rerun on this batch.

The results are shown in Table 11. Four points can be observed.

First, across the seven levels, $\mathrm{Cl}/|R|$ fluctuates between 0.48 and 0.56 without monotone trend (highest at the first level, lowest at the last, and back above 0.53 at the fifth and sixth), and shows no upward trend. Because Cl contains only occupancies that are unfinished after t_{now}, and these make up only about 0.61 of the table in this batch, $\mathrm{Cl}/|R|$ can reach at most about 0.61, so it cannot tell whether the share approaches 1. With the unfinished occupancies as the denominator, $\mathrm{Cl}/|R^\circ|$ lies in 0.78–0.91 at all seven levels (0.91 at the smallest size, 0.78 at the largest, maximum over pairs 1.00), the same order of magnitude as the 92% in E6 and E7. Hence the impact-set share does not rise with instance size; it still covers 78%–91% of the unfinished occupancies, however, and is not small relative to the rewritable set.

Second, as $|R|$ grows by a factor of 2.7, the median runtime of R2 grows from 2.9 ms to 10.4 ms, a factor of 3.6, and remains on the order of ten milliseconds; seven levels are insufficient to tell how fast the runtime grows with size. The first round of rerouting (without scope escalation) is feasible in 69/70 pairs. The single failing pair ($10 \times 5 \times 5$, random seed 42) restored feasibility in round 3 of scope escalation after failure, with a total runtime of 12.0 ms and the same makespan as the first round; hence the final result is feasible in 70/70 pairs.

Third, the runtime ratio of the fastest global recomputation RD to R2 rises from 1.5× to 3.3×. The ratio is not monotone (the ratio computed per pair and averaged per level dips from $k = 6 \to 7$), but the two ends differ by 1.8, far more than any within-level interquartile range (at most 0.74). Hence the larger the instance, the larger the runtime advantage of bounded repair over global recomputation.

Fourth, against the global right shift, R2 wins 7–10 of the 10 pairs on $C_{\max}$ at each level (10/0/0 at each of the last three); with only 10 pairs per level, no trend with size can be established.

Conclusion: seven levels are insufficient to infer the trend at larger sizes, but within the range from $8 \times 4 \times 4$ to $20 \times 10 \times 10$, the impact-set share does not rise with instance size, R2 stays on the order of ten milliseconds, and its runtime advantage over global recomputation widens with size.

6.12 Reproduction on the external-layout batch

This subsection tests whether the conclusions of E1, E2, E3, and E5 depend on the layouts we constructed. Setup: we use layouts 2–6 published by Lyu et al. (the second batch of Section 6.1), with 10 random seeds per layout, 50 pairs in total, and otherwise the same settings as in E1–E3 and E5; E5 runs only on layouts 2 and 6, with random seeds $\{42, 7, 2024\}$ and budgets $\{0.2, 1, 2\}$ s, giving 18 budget points.

The results correspond item by item to those of the main batch. E1: the operation-precedence graph yields an empty scope in 50/50 pairs, while the original schedule is infeasible. E2: the impact set is closed under dependency edges (50/50), and occupancies outside the set are unchanged entry by entry (50/50). E3: R1 is feasible in 0/50 and R2 in 50/50. Impact-set share: computed as in E1, the per-layout medians of $\mathrm{Cl}/|R|$ lie in 0.438–0.547, against a pooled median of 0.571 over the 50 pairs of the main batch; the external batch is slightly lower, and both are about one half. E5: at 17 of the 18 budget points R0+ attains the better $C_{\max}$, at 1 point the two tie (layout 6, random seed 2024, budget 0.2 s), and at no point is R0+ worse; these points are included in the pooled analysis of Section 6.7, and there is still no crossover.

Conclusion: with the published layouts, the conclusions of E1, E2, E3, and E5 are unchanged, so the conclusions on the scope object (C1, C2) do not depend on the layouts we constructed.

6.13 Summary

We group the results of this section by the contribution they support; the limitations are collected in Section 7.1.

The scope object (C1, C2: on which object the scope is delineated, and whether the resulting set is uniquely determined) is supported by four groups of results. Under corridor blockage, the scope on the operation-precedence graph is empty although the original schedule is indeed infeasible (E1, 50/50 over 50 pairs). The reservation impact set is closed under dependency edges, and fields outside the set are unchanged entry by entry (E2, 50/50, 50/50). With the same rerouting procedure and only the scope object changed, the feasibility rate goes from 0/50 to 50/50 (E3). Across four disturbance types crossed with two scope objects, the A/B dichotomy holds without exception, and for Class A the reservation impact set always contains the scope obtained on the operation-precedence graph (E6). These results correspond to the structural statements of Propositions 1–3. Hence whether feasibility can be restored depends on the scope object, not on the rerouting procedure.

The benefit of scoping (C3: what this way of delineating the scope gains). In E7, even the fastest global recomputation RD is slower than bounded repair (4.16 vs. 2.74 ms), so the millisecond-scale response is not because the baselines ran too long but comes from single-pass rerouting. In E5, warm-started global rescheduling is no worse than bounded repair on $C_{\max}$ at every budget point, and relaxing the budget produces no crossover. Switching to RA, which does not restrict the scope to the impact set (it uses the same rerouting as R2 and likewise does not rewrite finished occupancies), leaves

runtime and $C_{\max}$ almost unchanged, yet raises the rewrite fraction from 0.540 to 0.604 (per pair 31/18/1, $p = 8.8 \times 10^{-7}$; Section 6.9). Hence the benefit of scoping on the impact set lies in less rewriting, not in speed or makespan.

The remaining results delimit where these conclusions apply: static indicators computed from the network alone cannot predict the impact-set size (E4); when the share of disturbed operations reaches 1, bounded repair is still feasible in every pair and runs on the order of ten milliseconds (Section 6.10); when the instance grows to 20 jobs and 10 machines, the impact-set share relative to the unfinished occupancies does not rise (E8); and with the layouts published by Lyu et al., the conclusions of E1, E2, E3, and E5 are unchanged (Section 6.12), so the conclusions on the scope object do not depend on the layouts we constructed.

7 Conclusion

This paper makes three contributions. **First** (C1), it identifies a class of failures that the operation-precedence graph cannot capture: corridor blockage changes no operation assignment, so the scope delineated by affected operations is empty, although the original schedule is indeed infeasible (all 50 pairs in E1, Section 6.2). **Second** (C2), it takes the reservation impact set obtained by STRCas the rescheduling scope: once the dependency relations are given, whether an occupancy belongs to the set is uniquely determined, and no dependency edge points from inside the set to outside it (satisfied in all 50 pairs in E2, Section 6.3). **Third** (C3), it provides a local recovery matched to this scope and separates the sources of the individual results: with the same rerouting procedure and only the scope object changed, the feasibility rate goes from 0/50 to 50/50 (Section 6.4), so feasibility depends on the scope object rather than on the quality of the rerouting procedure.

Yielding relations, rerouting inside the set while keeping everything outside it as planned, and scope escalation after failure are all taken from the existing literature (Section 2.5). Our claim is as follows. By definition, the operation-precedence graph contains no yielding relations between corridor occupancies, and therefore cannot capture disturbances that invalidate only occupancies. Once the scope is delineated on the reservation dependency graph, the occupancies that must be rewritten are uniquely determined by the given precedence relations, not by a heuristic radius.

If the rewritable set is changed from the reservation impact set to "all occupancies unfinished after t_{now}" with all else unchanged, runtime and makespan remain almost the same, yet the rewrite fraction rises from 0.540 to 0.604 (Section 6.9); the smaller the impact set is relative to all unfinished occupancies, the larger this difference, and it vanishes as the two approach each other. Hence the millisecond-scale response comes from single-pass rerouting without restarting the search; **scoping on the impact set serves to reduce rewriting: occupancies outside the set are kept as planned (Proposition 3)**.

7.1 Limitations

The following four limitations are ordered by their impact on the preceding results; Section 7.2 gives a direction for addressing each of them.

(1) **Makespan.** Bounded repair trades makespan for runtime: over the 12 instance–seed pairs tested, warm-started global rescheduling is no worse than bounded repair on $C_{\max}$, and relaxing the budget produces no crossover (Section 6.7). This limitation concerns the benefit of the scoping and does not affect the conclusions on the scope object (C1, C2).

(2) **Guarantee and size of the impact set.** Proposition 3 gives only a sufficient condition for occupancies outside the set to remain unchanged, and the completeness of the dependency edge set has not been verified independently

and exhaustively. Nor is the impact set the minimal set needed to restore feasibility: it covers about 92% of the unfinished occupancies, fewer than one tenth less than including all of them in the rewrite, so the reduction in rewriting from scoping on the impact set is limited, and the closer the impact set is to all unfinished occupancies, the smaller this benefit. Moreover, static indicators computed from the network alone cannot predict the impact-set size (Section 6.6). This limitation concerns the strength of the guarantee in C2 and the size of the impact set; it does not affect the conclusion of C1.

(3) **Problem scope covered by the implementation.** Class B disturbances are handled by rerouting only; Class A additionally allows vehicle swapping and reassignment, but no change of processing order; and, by Assumption A3, each disturbance receives a single response. Opening up the processing order degenerates into global rescheduling, at which point "bounded" loses its meaning; several disturbances arriving one after another, estimation errors in disturbance information, and repeated rescheduling as new information arrives are not addressed. The instance size has been extended to 20 jobs and 10 machines without the impact-set share rising, but E8 has only seven levels, which is insufficient to infer the trend at larger sizes. Extrapolation of the results to larger instances and to continual disturbances therefore remains limited.

(4) **External reference points.** This line of research currently has no public benchmark that can be substituted in. The public instance families [2, 3] publish transport times as machine-to-machine travel-time matrices rather than as networks: we checked all 11 layouts of the two families one by one, and none can be reconstructed as an undirected corridor graph consistent with the published matrix, so the occupancy table R, the reservation dependency graph G_R, the reservation impact set $\mathrm{Cl}(\cdot)$, and Class B disturbances cannot be defined on them (Section 6.1). The layouts of the second batch are transcribed from the appendix of Lyu et al. [9], but the per-leg travel times and job data are still set by us, so the batch cannot be used to reproduce the reference values of Lyu et al. or van Os [23], and its coverage is smaller than that of the main batch. Hence only the networks of our instances come from the published literature, and the external validity of the results is limited; the relative comparisons among methods within the same implementation are unaffected.

7.2 Future work

The following four directions correspond one to one to the four limitations of the previous section.

1. Narrowing the makespan gap (Limitation 1). In the main batch, neither the decoding of R0+ nor that of RD rewrites finished occupancies (0 times over the 18 budget points of E5 and over the 50 pairs of E7), so the makespan gap between the two methods does not come from rewriting finished occupancies. Narrowing the gap would require limited reassignment or resequencing without rewriting finished occupancies, but this would make the method degenerate toward global rescheduling.

2. Verifying the completeness of the dependency edge set, estimating the gap between the impact set and a minimal set, and predicting its size (Limitation 2). *Completeness*: under the current yield edge set, the leakage found by the probe is 0. The next step is to probe with other disturbance forms (e.g., shortening occupancies, or delaying them by more than 1 time unit), rather than to keep adding dependency edges of the same kind. *Gap to a minimal set*: define the minimal rewritable set for a fixed repair capability and quantify the gap between the impact set and it; the current impact set covers about 92% of the unfinished occupancies, so the gap may be considerable. *Size characterization*: there is currently only one post hoc rule (the rewritable share, Figure 8), which varies with the schedule and cannot be computed before the disturbance; static indicators computed from the network alone are constant on practical layouts,

while changing only the machine configuration on the same network already changes the impact-set share (Section 6.6). A next step is to examine schedule-dependent quantities, such as the number of mutually overlapping occupancies on the disturbed corridor near t_{now}, and to predict the impact-set size from a given layout together with the current schedule.

3. Relaxing Assumption A3 (Limitation 3; A3 stipulates a single, fully observed disturbance). The next step is to relax this assumption and examine whether the impact set grows cumulatively under continual disturbances.

4. Travel times from an external source (Limitation 4). The layouts already come from the published literature, but the per-leg travel times are still set by us. The next step requires an instance family that publishes both a network and per-leg travel times; the non-uniform edge-weight sweep in E3 only tests the robustness of the results to the edge-weight setting and cannot replace externally published travel times.

References

[1] Andrea D'Ariano, Dario Pacciarelli, and Marco Pranzo. 2007. A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research* 183, 2 (2007), 643–657. doi:10.1016/j.ejor.2006.10.034

[2] Dalila B. M. M. Fontes, S. Mahdi Homayouni, and José F. Gonçalves. 2023. A hybrid particle swarm optimization and simulated annealing algorithm for the job shop scheduling problem with transport resources. *European Journal of Operational Research* 306, 3 (2023), 1140–1157. doi:10.1016/j.ejor.2022.09.006

[3] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6

[4] Ahmad Reza Jafarian-Moghaddam, Ali Esmailpoor Sanjabi, and Elham Mortazavi Moghaddam. 2026. Integrated tri-objective scheduling and internal logistics optimization in U-shaped flexible flow shops: a mathematical model and metaheuristic framework. *International Journal of Systems Science: Operations & Logistics* 13, 1 (2026), 2714000. doi:10.1080/23302674.2026.2714000

[5] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139

[6] Jürgen Kuster, Dietmar Jannach, and Gerhard Friedrich. 2007. Local Rescheduling: A novel approach for efficient response to schedule disruptions. In *2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched)*. IEEE, Honolulu, HI, USA, 79–86. doi:10.1109/SCIS.2007.367673

[7] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543

[8] Sirui Li, Wenbin Ouyang, Yining Ma, and Cathy Wu. 2025. Learning-guided rolling horizon optimization for long-horizon flexible job-shop scheduling. arXiv:2502.15791 [cs.LG] doi:10.48550/arXiv.2502.15791

[9] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109

[10] Nir Malka, Guy Shani, and Roni Stern. 2024. Online Planning for Multi Agent Path Finding in Inaccurate Maps. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Abu Dhabi, United Arab Emirates, 10214–10221. doi:10.1109/IROS58592.2024.10801975

[11] François Margot, Maurice Queyranne, and Yaoguang Wang. 2003. Decompositions, network flows, and a precedence constrained single-machine scheduling problem. *Operations Research* 51, 6 (2003), 981–992. doi:10.1287/opre.51.6.981.24912

[12] Alessandro Mascis and Dario Pacciarelli. 2002. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research* 143, 3 (2002), 498–517. doi:10.1016/S0377-2217(01)00338-1

[13] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355

[14] Djamila Ouelhadj and Sanja Petrovic. 2009. A Survey of Dynamic Scheduling in Manufacturing Systems. *Journal of Scheduling* 12, 4 (2009), 417–431. doi:10.1007/s10951-008-0090-8

[15] Fei Qiao, Li Li, Yumin Ma, and Bin Shi. 2012. Single Machine Oriented Match-Up Rescheduling Method for Semiconductor Manufacturing System. In *Intelligent Robotics and Applications (ICIRA 2012) (Lecture Notes in Computer Science, Vol. 7507)*. Springer, Berlin, Heidelberg, 217–226. doi:10.1007/978-3-642-33515-0_22

[16] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352

[17] Miguel A. Salido, Joan Escamilla, Federico Barber, and Adriana Giret. 2017. Rescheduling in job-shop problems for sustainable manufacturing systems. *Journal of Cleaner Production* 162 (2017), S121–S132. doi:10.1016/j.jclepro.2016.11.002

[18] Velusamy Subramaniam and Amritpal Singh Raheja. 2003. mAOR: A heuristic-based reactive repair mechanism for job shop schedules. *The International Journal of Advanced Manufacturing Technology* 22, 9-10 (2003), 669–680. doi:10.1007/s00170-003-1601-6

[19] Yong Jin Suh and Jin Young Choi. 2022. Efficient Fab Facility Layout with Spine Structure Using Genetic Algorithm under Various Material-Handling Considerations. *International Journal of Production Research* 60, 9 (2022), 2816–2829. doi:10.1080/00207543.2021.1904159

[20] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526

[21] Jiří Švancara, Marek Vlk, Roni Stern, Dor Atzmon, and Roman Barták. 2019. Online Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. AAAI Press, Honolulu, HI, USA, 7732–7739. doi:10.1609/aaai.v33i01.33017732

[22] Hongtao Tang, Xuewei Zhang, Jiawei Huang, Xuesong Xu, and Jiansha Lu. 2026. Integrated Scheduling of Multi-Objective Lot-Streaming Hybrid Flowshop with AGV under Dynamic Environments. *International Journal of Industrial Engineering Computations* 17, 2 (2026), 295–316. doi:10.5267/j.ijiec.2025.9.003

[23] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342

[24] Guilherme E Vieira, Jeffrey W Herrmann, and Edward Lin. 2003. Rescheduling Manufacturing Systems: A Framework of Strategies, Policies, and Methods. *Journal of Scheduling* 6, 1 (2003), 39–62. doi:10.1023/A:1022235519958

[25] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795

[26] Hongliang Zhang, Chaoqun Qin, Wenhui Zhang, Zhenxing Xu, Gongjie Xu, and Zhenhua Gao. 2023. Energy-Saving Scheduling for Flexible Job Shop Problem with AGV Transportation Considering Emergencies. *Systems* 11, 2 (2023), 103. doi:10.3390/systems11020103

[27] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, Kate Larson (Ed.). International Joint Conferences on Artificial Intelligence Organization, Jeju, South Korea, 311–320. doi:10.24963/ijcai.2024/35